

Grundlagen der Informatik (GDI)

Vorlesung – Hochschule Mannheim

Einführung in Java



Prof. Thomas Smits

Diese Materialien sind ausschließlich für den persönlichen Gebrauch durch Besucher der Vorlesung Grundlagen der Informatik (GDI) an der Technischen Hochschule Mannheim gedacht. Jede andere Nutzung ist ohne vorherige Zustimmung des Autors nicht zulässig.

Stand 2025-11-19

Technische Hochschule
mannheim

Inhaltsverzeichnis

1	Entwicklung der Programmierung	1
1.1	Ansätze zur Programmierung [5]	1
1.2	Programmiersprachen [7]	2
1.3	Maschinensprache [8]	2
1.4	Assemblersprachen [9]	3
1.5	Hochsprachen [10]	3
1.6	FORTRAN [11]	4
1.7	LISP [12]	4
1.8	COBOL [13]	5
1.9	ALGOL 60 [14]	5
1.10	BASIC [15]	6
1.11	Pascal [16]	6
1.12	C [17]	7
1.13	C++ [18]	8
1.14	Java [19]	8
2	Grundlagen	9
2.1	Entwicklung eines Java-Programms [21]	9
2.2	Java-VM [23]	10
2.3	Programmgerüst [24]	11
2.4	Ausgabeeweisung [26]	11
2.5	Eingabeeweisung [27]	12
2.6	Beispiel: Eingabeeweisung [28]	12
2.7	Erstes Java-Programm: Voraussetzungen [29]	13
3	Variablen	14
3.1	Motivation (noch einmal der Hamster) [31]	14
3.2	Motivation [32]	14
3.3	Definition von Variablen [33]	15
3.4	Berechnung von Werten [34]	15
3.5	Ausdrücke [35]	16
3.6	Lösung des Hamsterproblems [36]	16
3.7	Variable [37]	16
3.8	Was ist eine Variable? [38]	17
3.9	Variablennamen sind Bezeichner [39]	17

3.10	Konvention für Variablennamen [40]	18
3.11	Verwendung von Variablen [42]	19
3.12	Beispiel: Verwendung von Variablen [44]	19
3.13	Deklaration einer Variable [45]	20
3.14	Zuweisung [47]	21
3.15	Undefinierte Variablen [48]	22
3.16	Initialisierung von Variablen [49]	23
3.17	Beispiel: Initialisierung von Variablen [50]	23
4	Operatoren	24
4.1	Ausdruck [52]	24
4.2	Arten von Operatoren [53]	24
4.3	Syntax: Ausdruck [54]	25
4.4	Rangfolge der Operatoren [55]	26
5	Elementare Datentypen	27
5.1	Überblick über die elementaren Datentypen [57]	27
5.2	Datentyp boolean [58]	27
5.3	Logische Operatoren [59]	28
5.4	Kurzschlussoperatoren [60]	28
5.5	Logische Vergleichsoperatoren [61]	29
5.6	Rangfolge der logischen Operatoren [62]	29
5.7	Datentypen für Ganzzahlen [63]	30
5.8	Literale für Ganzzahlen [65]	31
5.9	Arithmetische Operatoren für ganze Zahlen [67]	32
5.10	Zusammengefasste Operatoren [68]	32
5.11	Inkrement und Dekrement-Operator [69]	33
5.12	Vergleichsoperatoren für ganze Zahlen [71]	34
5.13	Datentyp char [72]	34
5.14	Zeichenliterale [73]	34
5.15	Rechnen mit Zeichen [75]	35
5.16	Datentypen float und double [76]	35
5.17	Literale für float und double [77]	36
5.18	Arithmetische Operatoren für Fließkommazahlen [78]	36
5.19	Zusammengefasste Operatoren [79]	36
5.20	Besonderheit bei Fließkommazahlen [80]	37
5.21	Vergleichsoperatoren für Fließkommazahlen [81]	37
5.22	Vergleich zweier Fließkommazahlen mit == [82]	37
5.23	Konvertierung von Datentypen [83]	38
5.24	Cast-Operator [84]	38
6	Verbund (Java Klasse)	39
6.1	Motivation [87]	39
6.2	Verbund/Struktur [88]	39
6.3	Verbund/Struktur mit Klassen [89]	39

6.4	Beispiel: Verbund mit Java Klasse [90]	40
6.5	UML-Darstellung einer Klasse [91]	40
6.6	Speicherung einer Klasse [92]	40
6.7	Verwenden einer Klasse [93]	40
6.8	Referenzdatentypen [94]	41
6.9	Definition von Referenzdatentypen [96]	41
6.10	Referenzdatentypen [97]	42
6.11	Besonderer Wert: null [98]	42
6.12	Klasse und Objekt [99]	42
6.13	Zugriff auf Elemente [100]	43
6.14	Verschachtelte Definitionen [101]	43
6.15	Standardwerte [104]	44
6.16	Verkettung von Objekten [105]	44
7	Arrays	46
7.1	Typische Probleme [109]	46
7.2	Array [110]	46
7.3	Deklaration von Arrays [113]	47
7.4	Array im Speicher [115]	48
7.5	Initialisierung von Arrays [116]	48
7.6	Mehrdimensionale Arrays [118]	49
8	Strings	50
8.1	Motivation [122]	50
8.2	String [123]	50
8.3	String-Literale [124]	51
8.4	String als Referenzdatentyp [126]	51
8.5	Verkettung von Strings [127]	52
8.6	Methoden von String [129]	52
8.7	Vergleich von Strings [131]	53
9	Konstanten	54
9.1	Wozu Konstanten? [133]	54
9.2	Konstanten [134]	55
9.3	Beispiel: Innerhalb der Methode [136]	55
9.4	Beispiel: Außerhalb der Methode [137]	56
10	Auswahlanweisungen	57
10.1	Beispielhafte Probleme [139]	57
10.2	Ein- und Zweiseitige Auswahlanweisung [140]	57
10.3	Einseitige Auswahl in Java (if) [142]	58
10.4	Zweiseitige Auswahl in Java (if) [143]	58
10.5	Konventionen für Formatierung von if [144]	58
10.6	Mehrfachverzweigung mit if-else-if [145]	59
10.7	Mehrfachverzweigung mit switch [148]	60
10.8	Konventionen für Formatierung von switch [150]	61

10.9	Fragezeichen-Operator [151]	61
10.10	Beispiel für Fragezeichenoperator: Absolutbetrag [152]	62
11	Schleifen	63
11.1	Beispielhafte Probleme [154]	63
11.2	Schleife [155]	63
11.3	Arten von Schleifen [156]	63
11.4	while-Schleife [159]	64
11.5	Beispiel: while-Schleife [160]	65
11.6	do-while-Schleife [161]	65
11.7	Beispiel: do-while-Schleife [162]	66
11.8	for-Schleife [163]	66
11.9	Beispiel: for-Schleife [165]	68
11.10	foreach-Schleife [166]	68
11.11	Schleifenabbruch [169]	69
11.12	Codekonventionen [171]	70
12	Methoden (Prozeduren)	71
12.1	Unterprogramme und Funktionen [173]	71
12.2	Begriffe [174]	71
12.3	Methodendeklaration [175]	71
12.4	Arten von Methoden [177]	72
12.5	Signatur [178]	72
12.6	Return-Anweisung [179]	73
12.7	Regeln für die Return-Anweisung [181]	73
12.8	Methodenaufruf [185]	75
12.9	Pass-by-Value beim Methodenaufruf [187]	75
12.10	Dokumentation von Methoden [192]	76
12.11	Überladene Methoden [193]	77
12.12	Vararg-Methoden [196]	77
12.13	Lokale Variablen [198]	78
12.14	Sichtbarkeit von Variablen [200]	79
12.15	Parameter als Variablen [203]	80
12.16	Globale Variablen [204]	80
12.17	Seiteneffekt [207]	81
13	Rekursion	82
13.1	Idee der Rekursion [209]	82
13.2	Beispiel: Fakultätsberechnung [210]	82
13.3	Beispiel: Fibonacci-Zahlen [211]	83
14	Speicherverwaltung	84
14.1	Grundbegriffe [214]	84
14.2	Datenstruktur Stack [215]	84
14.3	Datenstruktur Heap [216]	84
14.4	Einsatz des Stacks [217]	85

14.5	Methodenaufruf und Stack [218]	85
14.6	Stack [219]	86
14.7	Stack und Heap [220]	86
14.8	Garbage Collection [222]	87

Kapitel 1

Entwicklung der Programmierung

1.1 Ansätze zur Programmierung [5]

Das **Programmierparadigma** legt den grundlegenden Stil fest, nach dem programmiert wird

Programmierparadigma

- **Imperative Programmierung**: Computer erhält Anweisungen, die das Problem schrittweise lösen

Imperative
Programmierung

1. Geh in den Keller.
2. Nimm das Bier aus dem Kasten.
3. Bring es mit nach oben.
4. Mach die Flasche auf und gib sie mir.

- **Deklarative Programmierung**: Anstatt des Lösungsweges wird das gewünschte Ergebnis beschrieben

Deklarative
Programmierung

1. Bier ist ein Getränk.
2. Alle Getränke sind im Keller in Kästen.
3. Getränke kann man aus dem Keller holen.
4. Vor dem Trinken muss man die Flasche öffnen.
5. Ich möchte ein Bier trinken.

Für die Programmierung von Computern gibt es verschiedene Ansätze, die man auch als *Programmierparadigma* bezeichnet. Die wichtigsten beiden sind die imperative und die deklarative Programmierung.

Bei der *imperativen Programmierung* besteht das Programm aus einer Abfolge von Anweisungen, die schrittweise zur Problemlösung führen. Es wird also *nicht* das Problem formuliert, sondern der Lösungsweg. Die bisher dargestellten Aktivitätsdiagramme sind ein Beispiel für eine imperative Problemlösung. Für die kommerzielle Softwareentwicklung hat die imperative Programmierung eine überragende Bedeutung. Auch bei den meisten Programmiersprachen handelt es sich um imperative Sprachen.

Die *deklarative Programmierung* verfolgt einen grundlegend anderen Ansatz. Hier wird das Problem beschrieben, mit allen Abhängigkeiten und Beziehungen, und der Computer leitet hieraus selbständig die Lösung ab. In der Forschung, vor allem im Bereich der *künstlichen*

Intelligenz und der *Wissensverarbeitung* kommt die deklarative Programmierung häufig zum Einsatz.

1.2 Programmiersprachen [7]

- Algorithmen müssen für Computer verständlich formuliert werden
- Computer sind dumm, daher muss die Formulierung
 - ▶ eindeutig
 - ▶ fehlerfrei
 - ▶ automatisiert verarbeitet werden können
- Natürliche Sprachen sind für Computer ungeeignet
 - ▶ nicht eindeutig
 - ▶ schwer zu interpretieren

Hilde winkte der Frau mit der blauen Mütze.

Programme für Computer werden in einer **Programmiersprache** (programming language) formuliert. Hierbei handelt es sich um spezielle Sprachen, die viel stärker formalisiert sind als die normale menschliche Sprache. Der Grund liegt darin, dass natürliche Sprachen viel zu schwer zu interpretieren sind und viel Wissen über den Kontext nötig ist, um sie zu verstehen. Da Computer dies nicht leisten können, sind Programmiersprachen stark formalisiert und eindeutig.

Programmiersprache

Es gibt eine ganze Reihe von Programmiersprachen, die im Laufe der Zeit entwickelt wurden. Obwohl wir uns in dieser Vorlesung ausschließlich mit Java beschäftigen werden, ist es sinnvoll zumindest ein Gefühl dafür zu entwickeln, wie viele verschiedene Sprachen es gibt. Da es in der Informatik üblich ist, beim Erlernen einer neuen Sprache als erstes ein Programm zu schreiben, dass „Hello World!“ ausgibt, wird dies auch in der folgenden Darstellung so gemacht.

1.3 Maschinensprache [8]

Maschinensprache ist die einzige Programmiersprache, die Prozessoren direkt verstehen

Maschinensprache

- Abfolge von Bits
- Für Menschen unverständlich und unlesbar

```
01010010 10100111 101001101 10101001
```



Mikroprozessoren sind in der Lage eine prozessorabhängige **Maschinensprache** auszuführen. Diese Sprache besteht aus einer Abfolge von Bits, die Funktionen des Prozessors direkt ansteuern, z. B. Lade ein Byte aus dem Hauptspeicher in Register AX, Addiere 5 zum Wert im Register,

Schreibe den Wert von Register AX wieder in den Hauptspeicher. Diese Programmiersprache ist für Menschen unverständlich und nicht zu benutzen.

1.4 Assemblersprachen [9]

Assemblersprachen liefern eine für Menschen lesbare Darstellung der Maschinensprache

Assemblersprachen

- Sprache wird von einem **Assemblierer** (**assembler**) in Maschinensprache übersetzt
- Erster Assembler 1948–1950 von Nathaniel Rochester programmiert

Assemblierer

```
.CONST
HW    DB    "Hallo Welt!$"
.CODE
.org 100h
start:
    MOV DX, OFFSET DGROUP:HW
    MOV AH, 09H
    INT 21H
    ret
end start
```



Bei Assemblersprachen handelt es sich um eine andere Darstellung der Maschinensprache des Prozessors, die für Menschen besser zu lesen ist. Den Bitfolgen der Maschinensprache werden Kürzel, **Mnemonic** genannt, zugeordnet. Außerdem besteht noch die Möglichkeit Kommentare und Sprungmarken zu verwenden.

Mnemonic

Das Programm in Assemblersprache wird dann von einem Werkzeug namens **Assembler** in die Maschinensprache des Prozessors umgewandelt. Bei dem Assembler handelt es sich nicht um einen **Compiler** wie er bei Hochsprachen verwendet wird, da hier keine Übersetzung von einer Sprache in eine andere stattfindet. Der Assembler und die Assemblersprache muss immer zu dem Prozessor passen, für den ein Programm übersetzt werden soll.

Assemblerprogramme sind wegen der geringen Abstraktionsebene aufwändig zu erstellen, laufen dafür aber auch mit der maximal möglichen Geschwindigkeit ab.

1.5 Hochsprachen [10]

Eine Programmiersprache, die von dem Computer und seiner internen Funktionsweise abstrahiert nennt man **höhere Programmiersprache** oder **Hochsprache** (**high-level programming language**)

höhere Programmiersprache
Hochsprache

- Computer kann sie nicht direkt ausführen
- Müssen für den Computer erst in Maschinensprache übersetzt werden
 - ▶ **Interpreter** führt das Programm in der Hochsprache schrittweise aus
 - ▶ **Compiler** übersetzt das Programm komplett in Maschinensprache, die dann ausgeführt wird

Interpreter

Compiler

- Alle gängigen Sprachen (abgesehen von Assembler) sind Hochsprachen

Da ein Prozessor nur Maschinensprache ausführen kann, müssen Programme in einer Hochsprache in die entsprechenden Maschinenbefehle übersetzt werden. Dies kann entweder zur Laufzeit durch einen Interpreter oder im Vorfeld durch einen Compiler erfolgen.

1.6 FORTRAN [11]

FORTRAN (FORmular TRANslator)

FORTRAN

- erste höhere Programmiersprache
- 1954 von John W. Backus entworfen
- Noch heute im Einsatz im Bereich wissenschaftlicher Berechnungen

```
program hallo
write(*,*) "Hallo Welt!"
end program hallo
```



Fortran gilt als die erste jemals tatsächlich realisierte höhere Programmiersprache. Sie geht zurück auf einen Vorschlag, den John W. Backus, Programmierer bei IBM, 1953 seinen Vorgesetzten unterbreitete.

Dem Entwurf der Sprache folgte die Entwicklung eines Compilers durch ein IBM-Team unter Leitung von Backus. Das Projekt begann 1954 und war ursprünglich auf sechs Monate ausgelegt. Tatsächlich konnte Harlan Herrick, der Erfinder der später heftig kritisierten Goto-Anweisung, am 20. September 1954 das erste Fortran-Programm ausführen. Doch erst 1957 wurde der Compiler für marktreif befunden und mit jedem IBM-704-System ausgeliefert. Backus hatte darauf bestanden, den Compiler von Anfang an mit der Fähigkeit zu Optimierungen auszustatten: Er sah voraus, dass sich Fortran nur dann durchsetzen würde, wenn ähnliche Ausführungsgeschwindigkeiten wie mit bisherigen Assembler-Programmen erzielt würden. Quelle: [Wikipedia](#)

1.7 LISP [12]

LISP (LISt Processor)

LISP

- Sprache für künstliche Intelligenz, noch heute weit verbreitet
- Sprache selbst ist sehr kompakt zu implementieren
- 1958 von John McCarthy entwickelt

```
(print "Hallo Welt!")
```



Lisp ist eine Familie von Programmiersprachen, die 1958 erstmals spezifiziert wurde und am Massachusetts Institute of Technology (MIT) in Anlehnung an den Lambda-Kalkül entstand. Es ist nach Fortran die zweitälteste Programmiersprache, die noch verbreitet ist.

Lisp steht für List Processing (Listen-Verarbeitung). Damit waren ursprünglich Fortran-Unterprogramme gemeint, mit denen symbolische Berechnungen durchgeführt werden sollten, wie sie im Lambda-Kalkül gebraucht werden. Steve Russell, einer der Studenten von John McCarthy, kam dann auf die fundamentale Idee, einen Interpreter für diese Ausdrücke zu programmieren, womit die Programmiersprache Lisp geboren war.

Quelle: [Wikipedia](#)

1.8 COBOL [13]

COBOL (COmmon Business Oriented Language)

COBOL

- Sprache für Geschäftsanwendungen
- 1959 von Grace Hopper entwickelt
- Immer noch relevant für Geschäftsanwendungen

```
000100 IDENTIFICATION DIVISION.  
000200 PROGRAM-ID. HELLOWORLD.  
000900 PROCEDURE DIVISION.  
001000 MAIN.  
001100 DISPLAY "Hallo Welt!".  
001200 STOP RUN.
```



COBOL ist eine Programmiersprache, die Ende der 1950er-Jahre, entstand und bis heute verwendet wird. Der Stil dieser Programmiersprache ist stark an die natürliche Sprache angelehnt.

COBOL entstand aus dem dringenden Wunsch, eine hardware-unabhängige standardisierte problemorientierte Sprache für die Erstellung von Programmen für den betriebswirtschaftlichen Bereich zu haben. Die Programmierung kaufmännischer Anwendungen unterscheidet sich von technisch-wissenschaftlichen Anwendungen durch die Handhabung großer Datenmengen statt der Ausführung umfangreicher Berechnungen. Nachdem die Programmierung technisch-wissenschaftlicher Anwendungen durch FORTRAN bereits sehr vereinfacht worden war, sollte die neue Programmiersprache die stärkere Berücksichtigung kaufmännischer Problemstellungen, insbesondere der Handhabung großer Datenmengen und ihrer Ein- und Ausgabe erreichen, die bis dahin weitgehend in Assemblersprachen programmiert wurden.

Quelle: [Wikipedia](#)

1.9 ALGOL 60 [14]

ALGOL 60 (ALGOrithmic Language)

ALGOL 60

- Von 1958–1963 von einem Gremium entwickelte Sprache

- Kommerziell nicht erfolgreich und heute nicht mehr im Einsatz
- Starker Einfluss auf andere Programmiersprachen (z. B. Pascal)

```
'COMMENT' HALLO, WELT PROGRAMM IN ALGOL 60;  
'BEGIN'  
    OUTSTRING(2, '(' HALLO, WELT ' ');  
'END'
```



1.10 BASIC [15]

BASIC (Beginner's All-purpose Symbolic Instruction Code)

BASIC

- 1964 von John G. Kemeny und Thomas E. Kurtz am Dartmouth College entwickelt
- Als Lehrsprache für Studenten entworfen
- Trotz vieler konzeptioneller Schwächen sehr erfolgreich bei den Homecomputern der 1980er und 1990er Jahre

```
10 PRINT "Hallo Welt!"
```



BASIC wurde 1964 von John G. Kemeny und Thomas E. Kurtz am Dartmouth College entwickelt, um den Elektrotechnikstudenten den Einstieg in die Programmierung gegenüber Algol und Fortran zu erleichtern. Am 1. Mai 1964 um vier Uhr Ortszeit, New Hampshire, liefen die ersten beiden BASIC-Programme simultan auf einem GE-225-Computer von General Electric im Keller des Dartmouth College.

BASIC wurde dann viele Jahre lang von immer neuen Informatikstudenten an diesem College weiterentwickelt, zudem propagierten es Kemeny und Kurtz ab den späten 1960er Jahren an mehreren Schulen der Gegend, die erstmals Computerkurse in ihr Unterrichtsprogramm aufnehmen wollten. BASIC war entsprechend dem Wunsch seiner „Väter“ für die Schulen kostenlos, im Gegensatz zu fast allen anderen damals üblichen Programmiersprachen, die meist mehrere tausend Dollar kosteten. Viele der damaligen großen Computerhersteller (wie etwa DEC) boten wegen der leichten Erlernbarkeit der Sprache und ihrer lizenzgebührenfreien Verwendbarkeit bald BASIC-Interpreter für ihre neuen Minicomputer an; viele mittelständische Unternehmen, die damals erstmals in größerer Zahl Computer anschafften, kamen so mit BASIC in Berührung.

Quelle: [Wikipedia](#)

1.11 Pascal [16]

- 1972 von Niklaus Wirth an der ETH Zürich entwickelt
- Als Lehrsprache für Studenten gedacht



```
program Hallo ( output );  
begin  
  writeln('Hallo Welt!');  
end.
```

Pascal ist eine Weiterentwicklung von Algol 60. Es lehnt sich in seiner Syntax stark an die englische Grammatik an. Dies soll die Lesbarkeit für Programmierneinsteiger verbessern und macht es daher besonders als Lehrsprache geeignet. Seine große Verbreitung in der professionellen Programmierung fand es als Borland/Turbo Pascal (später Object Pascal) – gegenüber dem Ur-Pascal wesentlich erweiterte und verbesserte Versionen.

Ein wichtiges Konzept, das Wirth zur Anwendung brachte, ist die starke Typisierung (engl. „strong typing“): Variablen sind bereits zur Übersetzungszeit einem bestimmten Datentyp zugeordnet, und dieser kann nicht nachträglich verändert werden. Typenstrenge bedeutet, dass Wertzuweisungen ausschließlich unter Variablen gleichen Typs erlaubt sind.

Quelle: [Wikipedia](#))

1.12 C [17]

- 1972 von Dennis Ritchie für die Programmierung des Betriebssystems Unix entwickelt
- Erlaubt eine maschinennahe Programmierung
- Starker Einfluss auf viele spätere Sprachen (C++, C#, Java)

```
#include <stdio.h>  
#include <stdlib.h>  
  
int main(void)  
{  
  puts("Hallo Welt!");  
}
```



C ist eine imperative Programmiersprache, die der Informatiker Dennis Ritchie in den frühen 1970er Jahren an den Bell Laboratories für die Systemprogrammierung des Betriebssystems Unix entwickelte. Seitdem ist sie auf vielen Computersystemen verbreitet.

Die Anwendungsbereiche von C sind sehr verschieden. Sie wird zur System- und Anwendungsprogrammierung eingesetzt. Die grundlegenden Programme aller Unix-Systeme und die Systemkernel vieler Betriebssysteme sind in C programmiert. Zahlreiche Sprachen, wie C++, Objective-C, C#, D, Java, PHP, Vala oder Perl orientieren sich an der Syntax und anderen Eigenschaften von C.

Quelle: [Wikipedia](#)

1.13 C++ [18]

- 1979 erweiterte Bjarne Stroustrup die Sprache C um Merkmale der Objektorientierung
- Heute noch sehr verbreitete Programmiersprache
- Starker Einfluss auf Java und C#

```
#include <iostream>
int main()
{
    std::cout << "Hallo Welt!" << std::endl;
}
```



C++ ist eine von der ISO genormte Programmiersprache. Sie wurde ab 1979 von Bjarne Stroustrup bei AT&T als Erweiterung der Programmiersprache C entwickelt. C++ ermöglicht sowohl die effiziente und maschinennahe Programmierung als auch eine Programmierung auf hohem Abstraktionsniveau.

Quelle: [Wikipedia](#)

1.14 Java [19]

- 1995 von James Gosling bei Sun Microsystems (heute Oracle) entwickelt
- Syntaktisch stark an C++ angelehnt
- Viele der komplexen Eigenschaften von C++ wurden nicht in Java übernommen

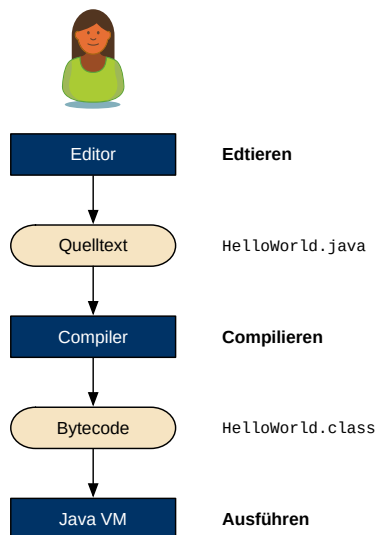
```
class Hallo {
    public static void main(String[] args) {
        System.out.println("Hallo Welt!");
    }
}
```



Kapitel 2

Grundlagen

2.1 Entwicklung eines Java-Programms [21]



- **Editieren** (edit): Eingabe des Programms
- Ergebnis: **Quelltext** (source code)
- **Compilieren** (compile): Übersetzen des Quelltexts in ein ausführbares Programm im Java-Bytecode durch den **Compiler** (javac)
- **Ausführen** (run): Ausführen des Programms auf der Java-VM (java)

Editieren
Quelltext
Compilieren
Compiler
Ausführen

```
$ javac HelloWorld.java  
$ java HelloWorld
```



Bei Programmiersprachen, die compiliert werden, also einen Compiler benutzen, besteht die Programmentwicklung aus drei Schritten:

Zuerst muss das Programm in den Computer eingegeben werden. Man spricht hier vom *editieren* und das Eingesetzte Werkzeug wird entsprechend als *Editor* bezeichnet. Nach dem Editieren liegt der Quelltext des Programms auf der Festplatte des Computers.

Da der Prozessor den Quelltext nicht ausführen kann, muss er erst vom *Compiler* in eine Form übersetzt werden, die der Computer ausführen kann. Hier gibt es bei Java eine Besonderheit, die es von anderen Programmiersprachen, wie z. B. C++ unterscheidet: Das Ergebnis der **Compilation** ist nicht ein Programm in der Maschinensprache des Prozessors, sondern der sogenannte **Java-Bytecode**. Der Grund hierfür ist, dass Java-Programme nicht direkt auf dem Prozessor des Computers ausgeführt werden, sondern auf einer *virtuellen Maschine* (VM) ablaufen. Der Bytecode ist somit die Maschinensprache der virtuellen Maschine. Der Name Bytecode stammt von der Tatsache, dass die Maschinenbefehle der Java-VM nur ein Byte (8 Bit) lang sind.

Nachdem der Bytecode vom Compiler erstellt wurde, kann er von der virtuellen Maschine ausgeführt werden.

Compilation
Java-Bytecode

2.2 Java-VM [23]

Java Programme laufen auf einer **Virtuellen Maschine** (Java-VM)

Virtuellen Maschine

- bietet einen „virtuelle Prozessor“ auf dem die Java-Programme ausgeführt werden
 - ▶ liest den Java-Bytecode ein
 - ▶ übersetzt ihn in den Maschinencode des Prozessors
 - ▶ lässt den Maschinencode ausführen
- ist selber in C/C++ geschrieben

Vorteil

- neuer Prozessor/neues Betriebssystem $\Rightarrow$ nur die Java-VM anpassen
- Java-Programme laufen weiterhin

Wie bereits erwähnt, laufen Java-Programme nicht direkt auf dem Prozessor des Computers, sondern auf einer virtuellen Maschine. Die virtuelle Maschine verhält sich wie ein *Interpreter*, der die Instruktionen des Bytecodes ausführt und somit in die Maschinensprache des Prozessors konvertiert.

Moderne Java-VMs (z. B. die Oracle HotSpot-VM) arbeiten nicht als reiner Interpreter, sondern sind in der Lage, den Bytecode bei Bedarf auf in die Maschinensprache des Prozessors zu compilieren. Hierdurch kann eine deutlich höhere Verarbeitungsgeschwindigkeit erreicht werden, als bei einem Interpreter. Da das Compilieren auch Zeit verbraucht, werden nur die häufig benutzten Programmteile (*hot spots*) übersetzt, der Rest wird interpretiert.

Der Grund für die Indirektion über eine Java-VM liegt darin, dass man hierdurch die Java-Programme unabhängig von der Hardware halten kann. Ein Java-Programm kann daher – vorausgesetzt es gibt eine passende VM – auf jeder beliebigen Hardware ausgeführt werden, ohne neu kompiliert werden zu müssen. Da es Java-VMs für nahezu jeden Computer und viele

Smartphones gibt, ist der Werbespruch von Java „Write once, run everywhere“ tatsächlich wahr.

2.3 Programmgerüst [24]

Ein Java-Programm besteht immer aus einer **Main-Klasse**

Main-Klasse

- Einzelne Syntaxelemente können leider erst später erklärt werden
- Vorläufig einfach nehmen wie es ist
- Java Programm gehört in die **Main-Methode**

Main-Methode

Syntax:

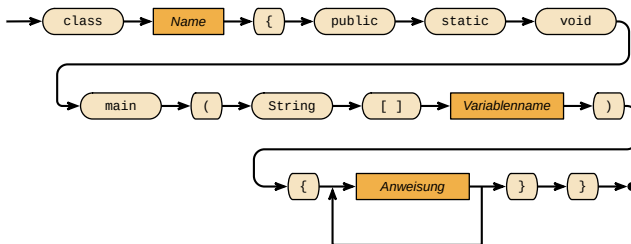
```
class NAME {

    public static void main(String[] args) {
        // Programm kommt hier hinein
    }
}
```



Auch hier gibt es einige Konstrukte (`class`, `public`, `static`, `[]`), die noch unbekannt sind. Diese werden im Laufe des Semesters erläutert und müssen vorläufig einfach hingenommen werden. Aus den Hamsterprogrammen sollte aber bereits die **Main-Methode** bekannt sein, die der Einstiegspunkt in das Java-Programm ist. Im Gegensatz zum Hamsterprogramm sind hier aber noch weitere Elemente nötig, damit das Programm syntaktisch korrekt wird.

Java-Programm



Syntax Java-Programm

2.4 Ausgabeeanweisung [26]

- Die **Ausgabeeanweisung** gibt Daten auf der Konsole aus
- Syntax:
`System.out.println(WERT);`

Ausgabeeanweisung

Ausgabeeanweisung



Syntax der Ausgabeeanweisung

```
System.out.println("Hallo Java"); // -> Hallo Java
System.out.println(7); // -> 7
System.out.println(39 + 3); // -> 42
```



Leider können wir an dieser Stelle noch nicht erläutern, wie die Ausgabeanweisung aufgebaut ist, da die entsprechende Theorie noch fehlt. Bis dies nachgeholt werden kann, sollte die Anweisung einfach wie sie ist in das jeweilige Programm übernommen werden. Da es sich bei Java nicht um eine reine Lehrsprache (wie z. B. Pascal) handelt, muss sich der Anfänger leider mit teilweise für ihn unnötig komplizierten Konstrukten herumschlagen.

2.5 Eingabeanweisung [27]

Die **Eingabeanweisung** liest Daten von der Konsole ein

Eingabeanweisung

1. Importieren der Klasse `Scanner`
`import java.util.Scanner;`
2. Anlegen einer Eingabe mit
`Scanner scanner = new Scanner(System.in);`
3. Lesen von Daten mit dem Scanner (abhängig von der Art der Daten)
 - Ganzzahlen
`int daten = scanner.nextInt();`
 - Fließkommazahlen
`double daten = scanner.nextDouble();`
 - Zeichenketten
`String daten = scanner.next();`

Die Verwendung der Eingabeanweisung ist noch etwas komplizierter als die Ausgabeanweisung. Bevor sie verwendet werden kann, muss sie dem Programm erst durch eine **Importanweisung** `import` bekannt gemacht werden. Auch hier können wir die Theorie erst viel später nachreichen.

Importanweisung

2.6 Beispiel: Eingabeanweisung [28]

```
import java.util.Scanner;

public class ConsoleIO {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        double wert = scanner.nextDouble();
    }
}
```



```
        System.out.println("Sie gaben ein: " + wert);  
    }  
}
```

2.7 Erstes Java-Programm: Voraussetzungen [29]

- **JDK** (Java Development Kit) Version 7
<http://www.oracle.com/technetwork/java/javase/downloads/index.html>
- *Texteditor*, z. B. NotePad++
<http://notepad-plus-plus.org/>

JDK

Weitere Werkzeuge

- Eclipse Standard 4.3
<http://www.eclipse.org/downloads/>

Für den weiteren Verlauf der Vorlesung werden wir Eclipse als Entwicklungsumgebung einsetzen. Es handelt sich hierbei um eine professionelle Entwicklungsumgebung, sodass sie für den Einsteiger erschreckend umfangreich erscheint. Da es sich aber um die am weitesten verbreitete Entwicklungsumgebung handelt, werden wir sie – trotz des unnötig großen Funktionsumfangs – in dieser Lehrveranstaltung einsetzen.

Kapitel 3

Variablen

3.1 Motivation (noch einmal der Hamster) [31]

Aufgabe: Der Hamster soll bis zur nächsten Wand laufen, dabei alle Körner einsammeln, die er findet, und an der Wand genauso viele Körner ablegen, wie er eingesammelt hat.

Notwendige Voraussetzungen:

- Gedächtnis (→ Speicher)
- Zählen/Rechnen/Vergleichen (→ Operationen)

Bei allen Konstrukten, die wir bisher kennengelernt haben, war es nicht nötig, Daten zu speichern. Um die hier gegebene Hamster-Aufgabe zu lösen, benötigen wir aber die Möglichkeit Daten abzulegen, nämlich die Anzahl der gesammelten Körner. Zusätzlich ist es nötig, dass wir mit diesen Daten auch rechnen können, denn nur so können wir laufend die Anzahl der bereits gesammelten Körner berechnen.

3.2 Motivation [32]

```
/* (1) "gefundene Anzahl ist 0" */
while (vornFrei()) {
    vor();
    while (kornDa()) {
        nimm();
        /* (2) "erhöhe gefundene Anzahl um 1" */
    }
}
while ( /* (3) "gefundene Anzahl größer als 0 ist" */ ) {
    gib();
    /* (4) "erniedrige gefundene Anzahl um 1" */
}
```



Dieses halbfertige Programm zeigt, an welchen Stellen wir mit den Daten umgehen müssen, um die gegebene Aufgabe zu lösen. Die markierten Stellen (1–4) müssen wir nun mit entsprechenden neuen Konstrukten auffüllen, damit das Programm die Hamster-Aufgabe löst.

3.3 Definition von Variablen [33]

Notwendig für (1)

- Festlegung eines Namens (→ **Variablendeklaration**)
- Reservierung von Speicherplatz (→ **Variablendefinition**)
- Festlegung des Typs der Daten (→ **Datentypen**)
- Initialen Wert festlegen (→ **Initialisierung**)
- hier: `int anzahl = 0;`

Variablendeklaration

Variablendefinition

Datentypen

Initialisierung

Aus der Mathematik ist das Konzept einer Variable bereits bekannt. Eine Variable kann in einer mathematischen Gleichung (z. B. $x^2 + 7x = 0$) für einen oder mehrere gesuchte Werte stehen oder in einer Funktion (z. B. $f(x) = x^3 + 2x^2$) einen beliebigen Wert (innerhalb des Definitionsbereiches der Funktion) annehmen.

Um die Hamster-Aufgabe zu lösen müssen wir der Variable einen Namen geben (*Deklaration*) und sie im Hauptspeicher ablegen (*Definition*). Variablen in Computerprogrammen brauchen zusätzlich einen ersten Startwert (*Initialisierung*) und haben einen sogenannten *Datentyp*, damit klar ist, wie der Computer mit der Variable umgehen soll.

3.4 Berechnung von Werten [34]

- **Ausdrücke** (**expressions**) erlauben die Berechnung von neuen Werten aus alten Werten
- Notwendig für (2), (3) und (4) im Beispiel
 - ▶ Operationen (→ Mathematik)
 - ▶ Operatoren
 - ▶ Operanden

Ausdrücke

Um mit den Variablen rechnen zu können verwenden wir *Ausdrücke*. Mit ihnen können wir aus bekannten Werten neue Werte berechnen. Auch dieses Konzept ist bereits aus der Mathematik bekannt, so ist z. B. $x + 8$ ein Ausdruck, der aus einem beliebigen Wert x einen neuen Wert, der um 8 größer ist berechnet. Ein Ausdruck besteht aus Operatoren und Operanden. Die Operatoren werden auf die Operanden angewendet, um ein neues Ergebnis zu erhalten. Im Ausdruck $x + 8$ ist $+$ der Operator und x und 8 sind die Operanden.

Wenn in einem Ausdruck eine Variable vorkommen (z. B. x), so wird die Variable durch ihren Wert ersetzt, bevor der Ausdruck ausgerechnet wird. Sei z. B. der folgende Ausdruck gegeben $(x + y) + 3 * x$ mit $x = 5$ und $y = 7$, so rechnet der Computer $(5 + 7) + 3 * 5 = 12 + 15 = 27$.

3.5 Ausdrücke [35]

Folgende Ausdrücke benötigen wir für die Lösung des Hamsterproblems

- Körner-Zähler um 1 erhöhen (2)
`anzahl = anzahl + 1;`
- Körner-Zähler um 1 erniedrigen (4)
`anzahl = anzahl - 1;`
- Prüfen, ob der Körner-Zähler größer als Null ist (3)
`anzahl > 0`

Die Ausdrücke sind selbsterklärend. Verwirrend könnte nur sein, dass dieselbe Variable links und rechts vom Gleichheitszeichen auftaucht. Das ist in der Programmierung üblich, in der Mathematik aber unbekannt. Einen Ausdruck `x = x + 7` muss man so lesen:

1. Der Ausdruck rechts vom Gleichheitszeichen (`x + 7`) wird ausgerechnet
2. Das Ergebnis wird der Variable zugewiesen

3.6 Lösung des Hamsterproblems [36]

```
void main() {  
    /* (1) "gefundene Anzahl ist 0" */  
    int anzahl = 0;  
    while (vornFrei()) {  
        vor();  
        while (kornDa()) {  
            nimm();  
            /* (2) "erhöhe gefundene Anzahl um 1" */  
            anzahl = anzahl + 1;  
        }  
    }  
  
    while (anzahl > 0) {  
        gib();  
        /* (4) "erniedrige gefundene Anzahl um 1" */  
        anzahl = anzahl - 1;  
    }  
}
```



3.7 Variable [37]

Mit einer **Variable** wird in einer Programmiersprache eine Speicherzelle verwaltet. Variablen müssen **deklariert** werden.

G. Büchel, Praktische Informatik

In der Speicherzelle wird ein **Wert** (value) abgelegt

Wert

Bisher haben wir die Mathematik bemüht, um das Konzept der **Variable** zu erläutern. Da sich aber Variablen in Computerprogrammen von denen in der Mathematik unterscheiden, soll hier noch einmal eine eigenständige Definition gegeben werden, die sich an der technischen Implementierung orientiert.

Prinzipiell dienen Variablen der Verwaltung von Speicherzellen im Hauptspeicher des Computers. Obwohl man theoretisch die Speicherzellen auch direkt ansprechen könnte, ist es deutlich komfortabler mit Variablen anstatt mit Speicherzellen zu arbeiten. Assemblerprogramme sind hier eine Ausnahme, da sie direkt über die Adressen auf den Speicher zugreifen und deswegen keine Variablen kennen. In allen Hochsprachen werden aber Variablen eingesetzt und in vielen Hochsprachen (so auch Java) ist ein direkter Zugriff auf den Speicher über Adressen nicht möglich. Sprachen wie C und C++ kennen aber tatsächlich beide Möglichkeiten: der Programmierer kann mit Variablen arbeiten, er hat aber auch die Möglichkeit direkt auf den Speicher zuzugreifen. Variablen sind also nichts anderes als eine Repräsentation einer Speicherstelle des Hauptspeichers im Programm.

Damit der Compiler (oder Interpreter) weiß, welche Speicherstellen ein Programm benötigt, müssen Variablen *deklariert* (d. h. bekannt gemacht) werden.

3.8 Was ist eine Variable? [38]

Variablen sind *Platzhalter* für konkreten Wert (analog zur Variable in der Mathematik)

- Programm funktioniert für beliebige Werte. Beispiel:
 - ▶ `bruttoPreis = nettoPreis * ( 1.0 + ustSatz / 100.0)`
 - ▶ `bruttoPreis`, `nettoPreis` und `ustSatz` sind Variablen
- Variablen können ihren *Wert beliebig oft ändern*
 - ▶ Gegensatz zur Mathematik!
 - ▶ Variable als temporärer Datenspeicher, z. B. für Zwischenergebnisse
Beispiel: `i = i + 1;`

Aus Sicht des Programmierers sind Variablen Platzhalter für Werte, die erst später (also nachdem das Programm fertiggestellt wurde) bestimmt werden. Mit Variablen kann man dann – wieder ähnlich zur Mathematik – mit den Platzhaltern anstatt mit den konkreten Werten arbeiten.

Ein wichtiger Unterschied zur Mathematik besteht darin, dass Variablen ihren Wert beliebig oft ändern können. Eine Variable kann also mehr als einmal *zugewiesen* werden. Dies ist in der Mathematik nicht möglich. Das Konstrukt `i = i + 1` wäre in einer mathematischen Formel nicht zulässig. Hier müsste man für jede Veränderung eine neue Variable einführen.

3.9 Variablennamen sind Bezeichner [39]

Variablennamen sind Bezeichner. Es gelten die Regeln für Bezeichner

- entspricht *nicht* einem Schlüsselwort
- enthält beliebige Unicode-Buchstaben, Zahlen, \$ oder _

- beginnt mit `_`, `$` oder einem Buchstaben
- kann beliebig lang werden

Gültige Bezeichner:

`_hugo`, `peter__$`, `antwort_42`, `meine7suenden`, `$rich$`, `haenselUndGretel`

Ungültige Bezeichner:

`7zwerge`, `goto`, `km/h`, `-strich`, `haensel&gretel`

Da jede Variable einen Namen bekommen muss (→ Deklaration), stellt sich die Frage, welche Namen zulässig sind. Variablennamen sind *Bezeichner*, sodass alle gültigen Bezeichner nach den o.g. Regeln als Variablennamen dienen können, um einen syntaktisch korrekten Variablennamen zu erhalten.

3.10 Konvention für Variablennamen [40]

Namenskonvention für Variablennamen werden nicht vom Compiler erzwungen aber freiwillig von allen Entwicklern eingehalten

Namenskonvention

Variablennamen

- beginnen mit Kleinbuchstabe
- weiter mit Kleinbuchstaben
- wenn ein neues Wort beginnt, wird ein Großbuchstabe verwendet (**Camel-Notation**)

Camel-Notation

Gut: `anzahlDatensaetze`, `limit`, `eingabeWert`

Schlecht: `anzahl_datensaetze`, `LIMIT`

Vernünftigerweise hält man sich an die folgenden Regeln, wenn man Variablennamen festlegt

- Namen aussagekräftig wählen
- Bedeutung sollte sofort klar werden
- Namen hinreichend lang wählen (ein und zwei Buchstaben vermeiden)
Ausnahme: Schleifenzähler
 - ▶ `bruttoPreis` statt `b`
 - ▶ `ustSatz` statt `m`
- entweder alle Variablennamen auf Deutsch oder alle auf Englisch

Syntaktisch korrekte Variablennamen können trotzdem sehr schlecht zu lesen sein, z. B. `__$__$` oder `$GaRGelK_arXXX__`. Daher haben sich Konventionen etabliert, wie man Variablen sinnvollerweise benennt. Diese Regeln werden nicht vom Compiler durchgesetzt und es obliegt daher Programmierer, sie einzuhalten.

Eine übersichtliche Benennung von Variablen und eine gute Formatierung des Quelltextes kann ganz erheblich zum Verständnis eines Programms beitragen.

3.11 Verwendung von Variablen [42]

- Ein Programm kann beliebig viele Variablen haben
 - ▶ für Eingabeparameter (`nettoPreis`)
 - ▶ für Konstanten (`ustSatz`)
 - ▶ für Zwischenwerte (`ustFaktor`)
 - ▶ für Ergebnisse (`bruttoPreis`)
- Jede Variable muss einen **Datentyp** (data type) haben
 - ▶ Wahrheitswert (`boolean`)
 - ▶ Ganze Zahl (`byte`, `short`, `int`, `long`)
 - ▶ Fließkommazahl (`float`, `double`)
 - ▶ Einzelnes Zeichen (`char`)
 - ▶ Zeichenkette String (`String`)
 - ▶ ...
- Variable wird im Hauptspeicher des Computers abgelegt

Datentyp

Die Anzahl von Variablen, die man in einem Programm verwenden kann, wird prinzipiell nur durch den Hauptspeicher des Computers begrenzt. Man kann also so viele Variablen anlegen, wie man für die Lösung seines Problems braucht. Da die Lesbarkeit eines Quelltextes auch ganz erheblich von der sinnvollen Verwendung von Variablen abhängt, sollte man auf keinen Fall Variablen für verschiedenen Zwecke wiederverwenden, um Speicherplatz zu sparen. Moderne Computer haben genug Speicher und die Klarheit des Quelltextes hat hier Vorrang.

Variablen haben in Computer immer einen sogenannten *Datentyp*, der festlegt, welche Art von Daten in einer Variable gespeichert werden können. Diese Festlegung ist nötig, da der Compiler wissen muss, wie viele Bytes er für die Speicherung der Daten im Hauptspeicher reservieren muss. Ein weiterer Vorteil von Datentypen besteht darin, dass der Compiler Fehler im Programm besser erkennen kann, die entstehen wenn man versucht, Variablen mit falschen Daten zu befüllen.

3.12 Beispiel: Verwendung von Variablen [44]

```
double ustSatz = 19.0;
double ustFaktor = 1.0 + ustSatz / 100.0;

Scanner scanner = new Scanner(System.in);
System.out.println("Bitte Nettopreis eingeben:");
double nettoPreis = scanner.nextDouble();

double bruttoPreis = nettoPreis * ustFaktor;

System.out.println("Der Bruttopreis ist: " + bruttoPreis);
```



Der Quelltext im Beispiel verwendet eine ganze Reihe von Variablen, um den Bruttopreis (also den Preis inklusive Umsatzsteuer) zu einem gegebenen Nettopreis zu berechnen. Die Syntax der verschiedenen Kommandos werden wir später noch behandelt. Hierzu wird eine Reihe von Variablen verwendet:

- `ustSatz` speichert den aktuellen Umsatzsteuersatz. Obwohl dieser Wert fix ist, wird er in einer Variable gehalten, damit man in späteren Schritten besser erkennen kann, wo der Wert eingeht.
- `ustFaktor` stellt den Multiplikationsfaktor dar, mit dem man den Nettopreis multiplizieren muss, um auf den Bruttopreis zu kommen.
- `scanner` ist Teil der Eingabeaufforderung.
- `nettoPreis` ist der vom Benutzer eingelesene Nettopreis.
- `bruttoPreis` ist der berechnete Bruttopreis.

Man hätte das Programm auch mit viel weniger Variablen schreiben können, die Lesbarkeit hätte aber erheblich darunter gelitten.

```
// Schlechtes Beispiel!
Scanner scanner = new Scanner(System.in);
System.out.println("Bitte Nettopreis eingeben:");
double nettoPreis = scanner.nextDouble();

System.out.println("Der Bruttopreis ist: "
    + nettoPreis * (1.0 + 19.0 / 100.0));
```



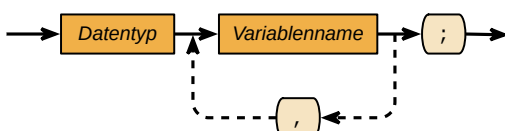
3.13 Deklaration einer Variable [45]

- **Deklaration:** **Name** (name) und **Datentyp** (type) einer Variable muss vor der ersten Verwendung dem Compiler bekannt gemacht werden
Syntax: `DATENTYP variablenname;`
- **Definition:** Für die Variable wird Speicher reserviert
 - ▶ In Java erfolgen Deklaration und Definition immer gleichzeitig und sind daher nicht zu unterscheiden
 - ▶ Manche Programmiersprachen erlauben eine Trennung von Deklaration und Definition (z. B. C und C++)

Deklaration
Name
Datentyp

Definition

Variablen-Deklaration



Deklaration einer Variablen

- Variablennamen müssen *eindeutig* sein

- ▶ Name nur einmal pro Programm verwenden
- ▶ Nur ein Datentyp pro Name
- Mehrere Variablen können auf einmal deklariert werden (schlechter Stil)

```
int antwort;  
int tick, trick, track; // schlecht!  
long vermoegenVonBillGates;  
  
double steuerSatz;  
  
String LieblingsEssen;  
String heimatOrt;
```



Es gibt Programmiersprachen, in denen die Trennung von Deklaration und Definition wichtig für die Semantik des Programms ist. So ist es in C z. B. manchmal nötig, eine globale Variable an mehreren Stellen zu deklarieren, damit die Programmteile Zugriff darauf erhalten, sie darf aber nur an einer einzigen Stelle definiert werden. Da Java vollständig auf das Konzept der globalen Variablen verzichtet, wird diese Unterscheidung nicht benötigt und Deklaration und Definition erfolgen immer in einem Schritt.

Da eine Variable eine Speicherstelle bezeichnet ist klar, dass man den Namen nicht mehrfach verwenden darf. Insofern muss der Name einer Variable eindeutig sein. Wir werden später noch sehen, dass diese Regel nur für bestimmte Teile eines Programms gilt und es durchaus möglich ist, an verschiedenen Stellen gleichnamige Variablen zu benutzen.

Die Deklaration von mehreren Variablen in einer Zeile gilt als schlechter Stil, weil sie unübersichtlich ist.

3.14 Zuweisung [47]

- Variablen erhalten ihren Wert durch eine **Zuweisung** (assignment)
- **Zuweisungsoperator** (assignment operator) = wird benutzt. Nicht mit dem Gleichheitszeichen der Mathematik zu verwechseln!
- Syntax: `Variablenname = Ausdruck;`

Zuweisung

Zuweisungsoperator

Wertzuweisung



Zuweisung einer Variablen

```
antwort = 42;  
vermoegenVonBillGates = 7000000000L;  
lieblingsEssen = "Spaghetti";
```



Damit eine Variable einen Wert speichern kann, muss ihr ein solcher zugewiesen werden. Hierzu gibt es einen *Operator* (Operatoren werden im nächsten Kapitel noch detailliert behandelt), der der Variable einen Wert zuweist also den Wert in die Variable hineinschreibt.

Ein häufiger Fehler von Programmierneulingen ist es, den Zuweisungsoperator = mit dem Gleichheitszeichen = aus der Mathematik zu verwechseln. Damit der Compiler die Zuweisung und den Vergleich unterscheiden kann, gibt es für den Vergleich einen anderen Operator ==, der schon eher dem Gleichheitszeichen der Mathematik entspricht. Die Operatoren werden später noch genauer behandelt.

- Zuweisung `a = b`: Variable `a` nach der Zuweisung den Wert von `b`.
- Vergleich `a == b`: Liefert einen booleschen Wert zurück, der `true` ist, wenn `a` denselben Wert wie `b` hat, andernfalls `false`.

3.15 Undefinierte Variablen [48]

- Neu deklarierte Variablen sind **undefiniert** (**undefined**), d. h. sie haben noch keinen Wert (**uninitialisierte Variable**)
 - ▶ würden zu Fehlern im Programm führen
 - ▶ werden vom Java-Compiler nicht akzeptiert
- **Initialisierung** Variable muss durch Zuweisung einen *ersten* Wert bekommen

undefiniert
uninitialisierte
Variable

Initialisierung

Fehlerhaftes Programm

```
int hatKeinenWert;  
System.out.println(hatKeinenWert); // Fehler
```



Fehlermeldung des Compilers

```
error: variable hatKeinenWert might not have been initialized  
    System.out.println(hatKeinenWert);
```

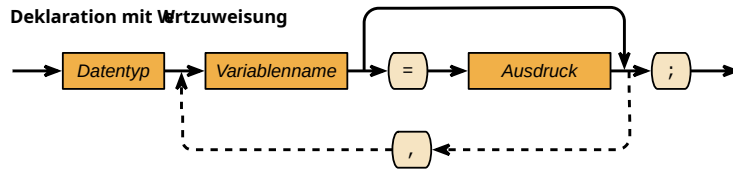


Bei der Deklaration der Variable wird Speicher entsprechend dem Datentyp der Variable reserviert. Der Inhalt des Speichers wird aber nicht verändert, d. h. es können beliebige Daten aus vorhergehenden Verwendungen des Speichers an dieser Stelle stehen. Damit ist der Inhalt der Variable nach der Deklaration nicht bekannt bzw. *undefiniert*. Der Begriff *undefiniert* wird bei Programmiersprachen immer dann verwendet, wenn für eine bestimmte Konstruktion die Semantik nicht im Rahmen der Programmiersprache definiert ist. Abhängig von der Programmiersprache kann dies bedeuten, dass das Programm einfach abstürzt oder mit falschen Werten rechnet. In Java hat man sich dafür entschieden, dass bereits der Compiler solche undefinierten Zustände entdeckt und die Übersetzung abbricht. Das hat den Vorteil, dass die Fehler bereits recht früh erkannt werden und nicht erst zur Laufzeit des Programms.

Die Variable muss also vor der ersten Verwendung durch eine Zuweisung einen Wert bekommen. Diese besondere Form der Zuweisung nennt man *Initialisierung*.

3.16 Initialisierung von Variablen [49]

- Variablen werden durch eine *Zuweisung* initialisiert
Syntax: `variablenname = AUSDRUCK;`
- Deklaration und Zuweisung können kombiniert werden
Syntax: `DATENTYP variablenname = AUSDRUCK;`



Syntax: Deklaration und Zuweisung einer Variablen

Häufig kombiniert man die Deklaration mit der Initialisierung, da sowieso jede Variable einen Wert bekommen muss und vorher nicht verwendet werden kann.

3.17 Beispiel: Initialisierung von Variablen [50]

```
int antwort; // Deklaration
antwort = 42; // Zuweisung

// Kombinierte Deklaration und Zuweisung
long vermoegeVonBillGates = 7000000000L;

double steuerSatz = 19.0;

// berechneter Ausdruck
double ustFaktor = 1.0 + (steuerSatz / 100.0);

String LieblingsEssen = "Spaghetti";
```



Kapitel 4

Operatoren

4.1 Ausdruck [52]

Ein **Ausdruck** (expression) verknüpft Operanden mit Hilfe eines Operators

Ausdruck

- **Operand** (operand) ist der Wert (Variablen oder Literale) der verknüpft werden soll
- **Operator** (operator) legt die Art der Verknüpfung fest (z. B. Addition mit +)

Operand

Operator

```
19 + 7  
vermoegenVonBillGates + 1000000
```



Variablen an sich sind nutzlos, solange man mit den Werten, die darin gespeichert sind nicht rechnen kann. Daher benötigt man, wie in der Mathematik *Operatoren*, die es erlauben aus bekannten Werten, neue Werte zu berechnen. Die Berechnung eines neuen Wertes erfolgt mit Hilfe eines *Ausdrucks*, z. B. $2 + 7$. Ein Ausdruck besteht aus

- *Operanden* die Werte mit einander verknüpfen (im Beispiel 2 und 7)
- *Operatoren* welche die Art der Verknüpfung festlegen (im Beispiel +)

4.2 Arten von Operatoren [53]

Drei Arten von Operatoren

- **Unäre Operatoren** haben nur einen Operanden
 - ▶ Syntax: `op Operand`
 - ▶ Beispiel: Negation einer Zahl ($a = -b$)
- **Binäre Operatoren** verknüpfen zwei Operanden
 - ▶ Syntax: `Operand1 op Operand2`
 - ▶ Beispiel: Multiplikation zweier Zahlen ($a = b * c$)
- **Ternäre Operatoren** verknüpfen drei Operanden (selten)
 - ▶ Syntax: `Operand1 op Operand2 op Operande3`

Unäre Operatoren

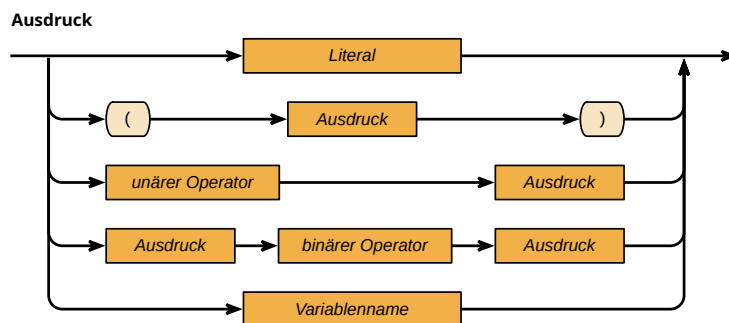
Binäre Operatoren

Ternäre Operatoren

- Beispiel: Bedingter Ausdruck ($x = a > b ? a : b$)

Man kann Operatoren danach klassifizieren, wie viele Operanden sie haben. Am häufigsten sind Operatoren mit zwei Operanden (*binäre Operatoren*). Deutlich seltener sind solche mit nur einem Operanden (*unäre Operatoren*). Mit drei Operanden (ternärer Operator) existiert in Java nur ein einziger Operator, der sogenannte *Fragezeichen-Operator*, der später noch behandelt werden wird.

4.3 Syntax: Ausdruck [54]



Syntax eines Ausdrucks

- *Literal* liefert Wert des Literals
- *Variablenname* liefert den aktuell in der Variablen gespeicherten Wert

Wie man an dem Syntaxdiagramm erkennen kann, können Ausdrücke selber wieder Ausdrücke enthalten. Dieses Vorgehen ist aus der Mathematik bekannt, wo man auch beliebige Ausdrücke verschachteln kann, z. B. $((3 + 5) : 7) + 6$. In diesem Beispiel liegen drei Ausdrücke vor, nämlich $A_1 = (3 + 5)$, $A_2 = A_1 : 7$ und $A_3 = A_2 + 6$, wobei jeweils der vorhergehende Ausdruck Teil des nächsten ist.

Die Reihenfolge der Ausführung kann man durch Klammern (siehe nächsten Abschnitt) steuern.

Operanden kommen auf drei Wegen in einen Ausdruck:

1. entweder als anderer Ausdruck,
2. als sog. **Literal**, das direkt einen Wert repräsentiert oder
3. als Variable bei der der Wert in der Variable in den Ausdruck eingeht.

Literal

Im ersten Fall wird der innere Ausdruck ausgewertet und das Ergebnis geht dann in den Gesamtausdruck ein. Im zweiten Fall steht das Ergebnis schon fest und im dritten Fall wird der in der Variable gespeicherte Wert in den Ausdruck eingesetzt.

4.4 Rangfolge der Operatoren [55]

- **Rangfolge (priority)** legt fest, welche Operatoren zuerst ausgewertet werden (vgl. „Punkt vor Strichrechnung“ in der Mathematik) Rangfolge
- **Assoziativität** legt fest, in welcher Richtung Operatoren mit gleicher Rangfolge ausgewertet werden Assoziativität
 - ▶ **links-assoziativ**: Auswertung erfolgt von Links nach Rechts links-assoziativ
 $3 + 5 + 6 \Leftrightarrow (3 + 5) + 6$
 - ▶ **rechts assoziativ**: Auswertung erfolgt von Rechts nach Links rechts assoziativ
 $a = b = c \Leftrightarrow a = (b = c)$
- Durch **Klammern** kann die Rangfolge geändert werden
 $(2 + 2) * 2 = 8$
- Im Zweifelsfall sollte man lieber Klammern setzen, als sich auf die Rangfolge zu verlassen:
 $2 + (2 * 2)$

Da häufig mehrere Operatoren aufeinandertreffen, muss man festlegen, welche Operatoren zuerst ausgewertet werden. Diese Reihenfolge wird durch die **Rangfolge** der Operatoren eindeutig bestimmt, d. h. für jeden Operator ist festgelegt, vor welchen anderen Operatoren er ausgewertet wird.

Wenn ein Ausdruck mehrere Operatoren mit identischer Rangfolge enthält, wird die Richtung der Ausführung durch die **Assoziativität** bestimmt.

Zum Beispiel sind Multiplikations- und Modulo-Operator linksassoziativ. Daher sind die beiden folgenden Anweisungen austauschbar:

```
int result = 5 * 70 % 6; // ergibt 2
int result = (5 * 70) % 6; // ergibt 2
```



Nicht aber

```
int result = 5 * (70 % 6); // ergibt 20
```



Rechtsassoziativ sind zum Beispiel die Vorzeichenoperatoren (+, -) oder die Zuweisung. Hier wird der Ausdruck von rechts nach links ausgewertet.

```
int summe;
int result = summe = 12; // summe ist 12, result ist 12
```



Kapitel 5

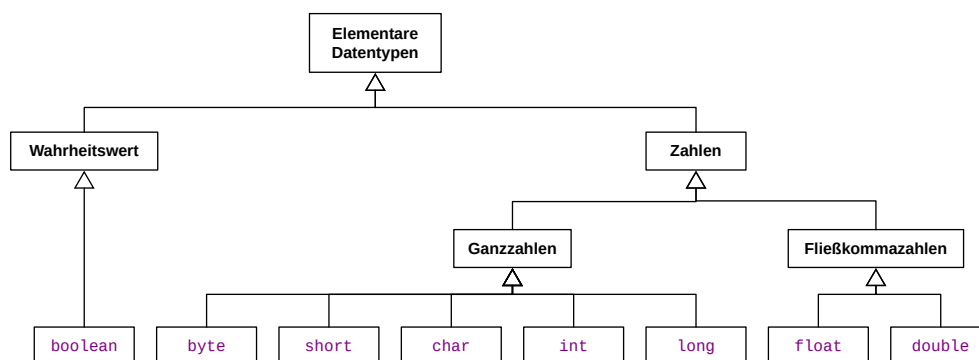
Elementare Datentypen

5.1 Überblick über die elementaren Datentypen [57]

Elementare Datentypen auch **primitive Datentypen** genannt

- Sind fest in die Sprache eingebaut (*Schlüsselworte*)
- Alle anderen Datentypen werden daraus zusammengesetzt

Elementare
Datentypen
primitive Datentypen



Elementare Datentypen

Die primitiven existieren in Java aus Performancegründen. Im Gegensatz zu anderen Sprachen (z. B. Ruby) hat man sich entschlossen, neben den klassenbasierten auch noch primitive Datentypen in Java zu verwenden. Da die primitiven Datentypen eine einfache Wertsemantik haben, kann man Operationen auf den Typen deutlich schneller ausführen als auf entsprechenden Klassentypen. Details hierzu werden später noch geliefert.

Wichtig zu verstehen ist, dass die primitiven Datentypen die Grundlage aller anderen Datentypen in Java sind. D. h. alle anderen Datentypen entstehen durch die Kombination der primitiven Datentypen.

5.2 Datentyp boolean [58]

Der Java-Datentyp **boolean** stellt einen Wahrheitswert dar

boolean

- Wertebereich: { wahr, falsch }
- Literale: **true** und **false**

- Kann Ergebnis eines logischen Ausdrucks speichern
- Variable kann in Bedingungen und Schleifen verwendet werden

```
boolean b2;
boolean b1;
b1 = 5 < 7;
b2 = b1 && (8 != 3) || (4 == 5);

if (b1) {
    System.out.println("b1 = true");
}
```



Der einfachste Datentyp in Java ist der Datentyp `boolean`. Er speichert einen einzigen Wahrheitswert und kann daher nur zwei Zustände darstellen: wahr (`true`) oder falsch (`false`).

5.3 Logische Operatoren [59]

- **Logische Operatoren** verknüpfen Wahrheitswerte miteinander
- Eingabe: zwei Wahrheitswerte (`true` oder `false`) bei `^`, `&&` und `||` bzw. ein Wahrheitswert bei `!`
- Ergebnis: eine Wahrheitswert (`true` oder `false`)

Logische Operatoren

Operator	Bedeutung	Beispiel	Ergebnis
<code>^</code>	exklusiv oder	<code>true ^ true</code>	<code>false</code>
<code>&&</code>	und	<code>true && false</code>	<code>false</code>
<code> </code>	oder	<code>true false</code>	<code>true</code>
<code>!</code>	nicht	<code>!false</code>	<code>true</code>

Die Eigenschaften der logischen Operatoren wurden bereits zu Beginn der Vorlesung im Rahmen der booleschen Algebra behandelt. Es handelt sich hier also nur um eine kurze Wiederholung.

5.4 Kurzschlussoperatoren [60]

`&&` und `||` sind sogenannte **Kurzschlussoperatoren**

Kurzschlussoperatoren

- `&&` und `||` brechen die Auswertung ab, sobald das Ergebnis feststeht
 - ▶ bei `&&` ist ein Ausdruck `false` $\Rightarrow$ gesamter Ausdruck ist `false`
 - ▶ bei `||` ist ein Ausdruck `true` $\Rightarrow$ gesamter Ausdruck ist `true`
- `&` und `|` werten den gesamten Ausdruck aus und berechnen dann erst das Ergebnis
- Normalerweise braucht man `&` und `|` nicht und sollte immer `&&` und `||` verwenden

Es gibt in Java zwei Arten von logischen Operatoren für Und und Oder, die normalen `|` und `&` und die so genannten Kurzschlussoperatoren `||` und `&&`. Bei den normalen Operatoren werden alle (Teil-)Ausdrücke ausgewertet und erst dann wird der logische Operator angewandt. Bei den Kurzschlussoperatoren wird die Auswertung streng von links nach rechts durchgeführt und sie wird abgebrochen sobald der logische Wert des Gesamtausdrucks feststeht.

Da ein Abbruch der Auswertung in nahezu allen Fällen im Interesse des Programmierers liegt, werden die normalen logischen Operatoren in dieser Vorlesung nicht verwendet und es kommen immer die Kurzschlussoperatoren zum Einsatz

5.5 Logische Vergleichsoperatoren [61]

- **Logische Vergleichsoperatoren** verknüpfen Wahrheitswerte miteinander
- Entsprechen den bekannten Operatoren aus der Mathematik
- Eingabe: zwei Wahrheitswerte
- Ergebnis: eine Wahrheitswert (**true** oder **false**)
- Häufig überflüssig

Logische Vergleichsoperatoren

Operator	Bedeutung	Beispiel	Ergebnis
<code>==</code>	gleich	<code>true == false</code>	<code>false</code>
<code>!=</code>	ungleich	<code>true != false</code>	<code>true</code>

Man kann Wahrheitswerte auch über Vergleichsoperatoren miteinander in Beziehung setzen, d. h. auf den Wert der Variable testen. Da die Eingabe zwei boolesche Ausdrücke sind und das Ergebnis ein boolescher Wert, braucht man die Operatoren nur selten, denn:

- `a == true` $\Leftrightarrow$ `a`
- `a == false` $\Leftrightarrow$ `!a`
- `a != true` $\Leftrightarrow$ `!a`
- `a != false` $\Leftrightarrow$ `a`

5.6 Rangfolge der logischen Operatoren [62]

Rang	Operator	Assoziativität
1.	<code>!</code>	R
2.	<code>==</code>	L
2.	<code>!=</code>	L
3.	<code>&&</code>	L
4.	<code> </code>	L
5.	<code>=</code>	R

Beispiel



```

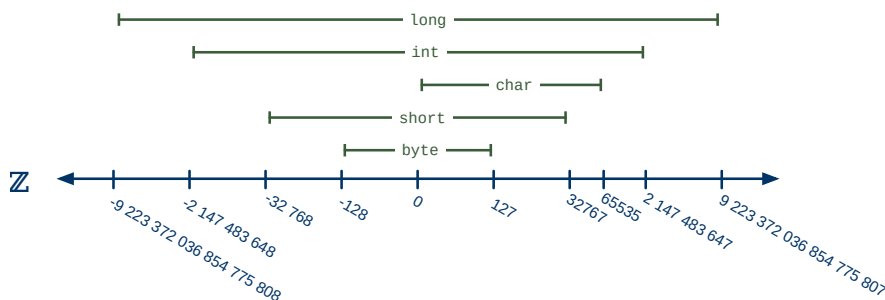
true && false == false || true && true
(true && (false == false)) || (true && true)

```

5.7 Datentypen für Ganzzahlen [63]

Ganzzahlen oder **Ordinalzahlen** (**integer**) haben einen endlichen Wertebereich (entsprechen einem Teil der ganzen Zahlen aus der Mathematik)

Ganzzahlen
Ordinalzahlen



Wertebereich der Ganzzahlen in Java

- **byte, short, int, long**: Speicherung von Zahlen mit Vorzeichen
- **char**: Ein Zeichen des Unicode-Zeichensatzes

Datentyp	Breite (Bit)	Wertebereich
byte	8	$-2^7 - 2^7 - 1$
short	16	$-2^{15} - 2^{15} - 1$
char	16	$0 - 2^{16} - 1$
int	32	$-2^{31} - 2^{31} - 1$
long	64	$-2^{63} - 2^{63} - 1$

Wichtig ist zu verstehen, dass die Ganzzahlen in einer Programmiersprache immer nur einen Teil der ganzen Zahlen aus der Mathematik abdecken können. Während die Zahlen in der Mathematik von Minus Unendlich bis Unendlich laufen, kann der Computer nur Zahlen einer bestimmten Größe speichern, da er sie auf Bitmuster endlicher Länge abbilden muss. Daher gibt es immer eine kleinste und größte Zahl, die man im Computer darstellen kann.

Um dem Programmierer eine möglichst platzsparende Speicherung der Zahlen zu gestatten, hat er die Wahl zwischen verschiedenen Datentypen, die sich in der Anzahl der verwendeten Bits und damit im Wertebereich unterscheiden. Am wenigsten Platz benötigt der Datentyp **byte** und am meisten **long**. Wichtig ist, für ein Problem einen Datentyp mit hinreichend großem Wertebereich zu wählen, da (wie bereits erläutert) die Werte überlaufen können.

Beispiel für einen Überlauf

```

byte b = 127;
b = b + 1;

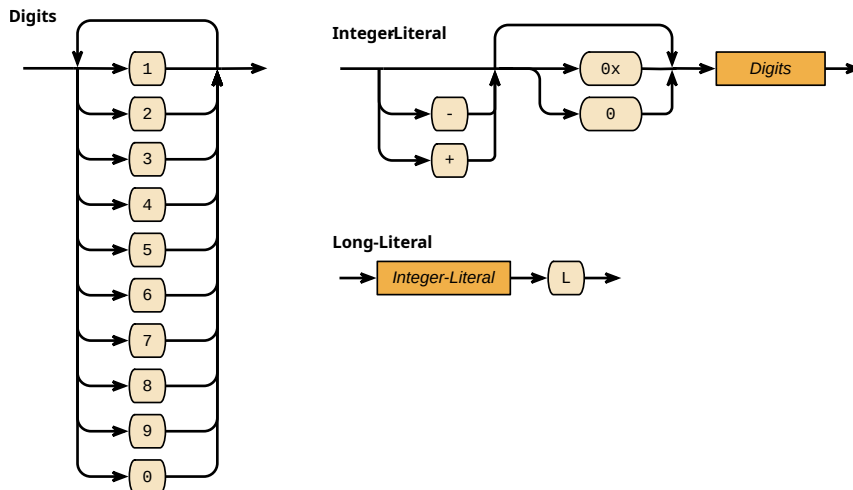
```



```
System.out.println(b); // -> -128
```

Der Datentyp `char` ist eine Besonderheit, da er sowohl zur Speicherung von Ganzzahlen als auch für Zeichen verwendet werden kann. Die Verwendung für Zahlen ist aber unüblich und sollte daher vermieden werden.

5.8 Literale für Ganzzahlen [65]



Syntax für Ganzzahl-Literale

■ Vorzeichen

- ▶ Positives Vorzeichen kann weggelassen werden: 476 oder +467
- ▶ Negatives Vorzeichen ohne Leerzeichen: -3521

■ Datentyp

- ▶ Standardmäßig hat das Literal den Typ `int`
- ▶ Für `long` muss ein `L` angehängt werden
7000000000L

■ Prefix

- ▶ `0` für Oktalzahlen
- ▶ `0x` für Hexadezimalzahlen
- ▶ `0b` für Binärzahlen
- ▶ ohne Prefix für Dezimalzahlen

Der Prefix für Oktalzahlen ist eine häufige Falle für Programmierneulinge. In der Mathematik ist es völlig egal, ob man 42 oder 042 schreibt. Beides bezeichnet dieselbe Zahl. In Java ist das anders, da durch die führende 0 eine Oktalzahl angezeigt wird. 042 entspricht der Dezimalzahl 34.

5.9 Arithmetische Operatoren für ganze Zahlen [67]

- *Arithmetische Operatoren* verknüpfen Zahlen miteinander (entsprechen den bekannten Operatoren aus der Mathematik)
- Eingabe: zwei ganze Zahlen (*byte*, *short*, *int*, *long*)
- Ergebnis: eine ganze Zahl (*int*, *long*)
- Division durch 0 führt zu einem Laufzeitfehler ($5 / 0$)

Operator	Bedeutung	Beispiel	Ergebnis
+	Addition	$39 + 3$	42
-	Subtraktion	$26 - 3$	23
*	Multiplikation	$19 * 7$	133
/	Division	$19 / 7$	2
%	Modulo	$19 \% 7$	5

Die arithmetischen Operatoren für ganze Zahlen sind alle bereits aus der Mathematik bekannt, werden nur teilweise durch andere Zeichen repräsentiert.

Der Modulo-Operator kommt in der Schulmathematik selten zum Einsatz, hat in der Informatik aber eine überragende Rolle, da viele Problemlösungen auf Module-Arithmetik beruhen. Details hierzu werden in anderen Veranstaltungen im Rahmen von Hashtabellen und ähnlichen Datenstrukturen behandelt werden.

5.10 Zusammengefasste Operatoren [68]

Zuweisung und Rechnung können zusammengefasst werden (häufig unübersichtlich)

Operator	Bedeutung
$a += b$	$a = a + b$
$a -= b$	$a = a - b$
$a *= b$	$a = a * b$
$a /= b$	$a = a / b$
$a \% = b$	$a = a \% b$

Aus C hat Java die Eigenschaft geerbt, dass man den Zuweisungsoperator mit den arithmetischen Operatoren kombinieren kann. Die so entstehenden Konstrukte sind häufig sehr unübersichtlich und bringen wenig für das Programm. Daher kann man die kombinierten Operatoren im Allgemeinen getrost ignorieren. Da aber andere Programmierer sie benutzen könnten, sollte man sie zumindest kennen, um den Quelltext verstehen zu können.

5.11 Inkrement und Dekrement-Operator [69]

Für die sehr häufige Operation des Erhöhen und Erniedrigen einer Zahl um den Wert 1 gibt es spezielle Operatoren (**Inkrement-Operator** und **Dekrement-Operator**)

- **Präfix-Operator** (++a und --a) verändern den Wert der Variable und geben diesen zurück
- **Postfix-Operator** (a++ und a--) geben den Wert der Variable zurück und verändern sie erst danach

Inkrement-Operator
Dekrement-Operator
Präfix-Operator

Postfix-Operator

Operator	Bedeutung
a++	a = a + 1
++a	a = a + 1
a--	a = a - 1
--a	a = a - 1

Eine der häufigsten arithmetischen Operationen ist das Erhöhen oder Erniedrigen einer ganzen Zahl um den Wert Eins. Daher gibt es für diesen Fall zwei (genaugenommen vier) Operatoren, nämlich ++ und --, die auf eine Variable (nicht auf einen Wert) angewendet werden können.

Eine für Einsteiger schwer verständliche Eigenschaft der Operatoren ist, welchen Wert der Ausdruck selber zurückliefert. In jedem Fall wird die Variable erhöht oder erniedrigt. Von der Position des Operators (vor oder hinter der Variable) hängt aber ab, welchen Wert der Ausdruck zurückgibt. Steht der Operator vor der Variable, wird zuerst die Variable verändert und dann wird das Ergebnis zurückgegeben. Steht der dahinter, wird erst der Wert der Variable zurückgegeben und dann der Wert erhöht.

Steht der Operator nicht in einer Zuweisung, ist es egal, welche Form man verwendet. Es hat sich aber bei den meisten Programmierern eingebürgert, dann die Postfix-Schreibweise (a++) zu verwenden.

```
int a = 7;
int b = a++;
System.out.println("a=" + a + ", b=" + b); // a=8, b=7
```



```
int a = 7;
int b = a--;
System.out.println("a=" + a + ", b=" + b); // a=6, b=7
```



```
int a = 7;
int b = ++a;
System.out.println("a=" + a + ", b=" + b); // a=8, b=8
```




```
int a = 7;
int b = --a;
System.out.println("a=" + a + ", b=" + b); // a=6, b=6
```

5.12 Vergleichsoperatoren für ganze Zahlen [71]

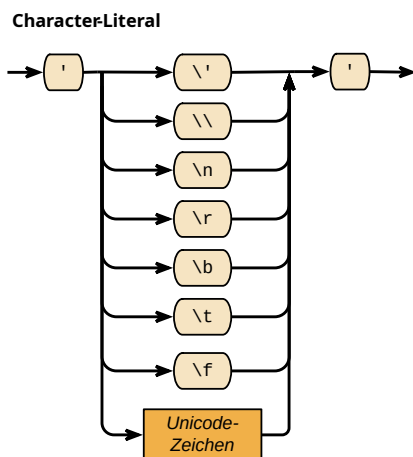
- *Numerische Vergleichsoperatoren* verknüpfen Zahlen miteinander
- Entsprechen den bekannten Operatoren aus der Mathematik
- Eingabe: zwei ganze Zahlen
- Ergebnis: ein Wahrheitswert (*true* oder *false*)

Operator	Bedeutung	Beispiel	Ergebnis
<	kleiner	8 < 9	<i>true</i>
>	größer	9 > 4	<i>true</i>
<=	kleiner gleich	7 <= 3	<i>false</i>
>=	größer gleich	7 >= 3	<i>true</i>
==	gleich	6 == 5	<i>false</i>
!=	ungleich	6 != 5	<i>true</i>

5.13 Datentyp char [72]

- Zeichen werden mit dem Datentyp *char* verwaltet
- Wertebereich von 0 – 65535
- Repräsentiert ein *einziges* Unicode-Zeichen
- Ist Grundlage für die Verwaltung von Zeichenketten (Strings)

5.14 Zeichenliterale [73]



Syntax für ein char-Literal

Zeichenlitterale repräsentieren ein einziges Zeichen, umschlossen von einfachen Anführungszeichen `'`

Bestimmte Sonderzeichen müssen **escaped**, d. h. speziell notiert, werden

escaped

Escape-Sequenz	Bedeutung
<code>\'</code>	Anführungszeichen <code>'</code>
<code>\\</code>	Backslash <code>\</code>
<code>\t</code>	Tabulator
<code>\n</code>	Zeilenvorschub (newline)
<code>\r</code>	Wagenrücklauf (carriage return)
<code>\b</code>	Backspace
<code>\f</code>	Seitenvorschub (form feed)

5.15 Rechnen mit Zeichen [75]

`char` kann als auch als Zahlenwert betrachtet werden und in Berechnungen und numerische Vergleiche eingehen

```
char a = 'A';
a++;
System.out.println(a); // -> B

char c = 'K';
if (('A' <= c) && (c <= 'Z')) {
    // Großbuchstabe A-Z
}
```

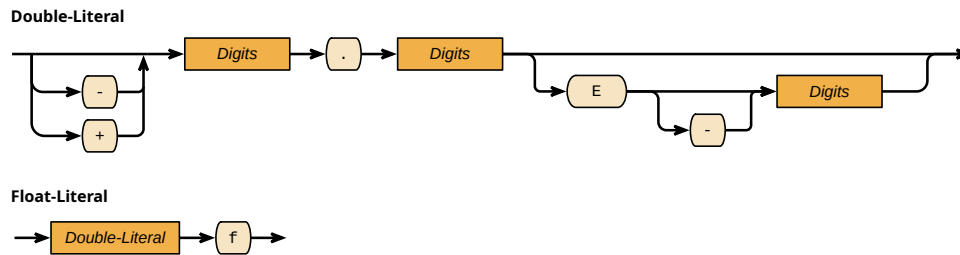


5.16 Datentypen float und double [76]

Typen für Fließkommazahlen in Java sind

- **double** – 64-Bit Fließkommazahl
 - ▶ Wertebereich $-1,79 \cdot 10^{308}$ – $1,79 \cdot 10^{308}$, maximal 15 Dezimalziffern
 - ▶ Literale mit Dezimalpunkt `.` und optionalem Exponenten `e`, z. B. `3.141`, `-0.02e-1` oder `6.23e23`
- **float** – 32-Bit Fließkommazahl
 - ▶ Wertebereich $-3,4 \cdot 10^{38}$ – $3,4 \cdot 10^{38}$, maximal 7 Dezimalziffern
 - ▶ Literale mit Dezimalpunkt `.` und `f` und optionalem Exponenten `e`, z. B. `3.141f`, `-0.02e-1f` oder `1.141e3f`

5.17 Literale für float und double [77]



Syntax für float- und double-Literale

5.18 Arithmetische Operatoren für Fließkommazahlen [78]

- *Arithmetische Operatoren* verknüpfen Zahlen miteinander
- Eingabe: zwei Zahlen (davon *mindestens* eine Fließkommazahl)
- Ergebnis: eine Fließkommazahl

Operator	Bedeutung	Beispiel	Ergebnis
+	Addition	39.1 + 3.3	42.4
-	Subtraktion	26.0 - 3	23.0
*	Multiplikation	19.3 * 7.8	150.54
/	Division	37.41 / 4.3	8.7
%	Modulo	19.0 % 7	5.0

5.19 Zusammengefasste Operatoren [79]

Zuweisung und Rechnung können auch bei Fließkommazahlen zusammengefasst werden (häufig unübersichtlich)

Operator	Bedeutung
<code>a += b</code>	<code>a = a + b</code>
<code>a -= b</code>	<code>a = a - b</code>
<code>a *= b</code>	<code>a = a * b</code>
<code>a /= b</code>	<code>a = a / b</code>
<code>a %= b</code>	<code>a = a % b</code>
<code>a++</code>	<code>a = a + 1</code>
<code>++a</code>	<code>a = a + 1</code>
<code>a--</code>	<code>a = a - 1</code>
<code>--a</code>	<code>a = a - 1</code>

5.20 Besonderheit bei Fließkommazahlen [80]

Division durch Null ist bei Fließkommazahlen erlaubt. Ergebnis **Infinity** und **NaN** (not a number)

Infinity
NaN

Beispiele

- `2 / 0.0 = +Infinity`
- `-2.0 / 0 = -Infinity`
- `0.0 / 0.0 = NaN`

5.21 Vergleichsoperatoren für Fließkommazahlen [81]

- Identisch zu den Operatoren für Ganzzahlen
- Entsprechen den bekannten Operatoren aus der Mathematik
- Eingabe: zwei Fließkommazahlen
- Ergebnis: ein Wahrheitswert (**true** oder **false**)

Operator	Bedeutung	Beispiel	Ergebnis
<code><</code>	kleiner	<code>8.0 < 9.0</code>	true
<code>></code>	größer	<code>9.0 > 4.0</code>	true
<code><=</code>	kleiner gleich	<code>7.0 <= 3.0</code>	false
<code>>=</code>	größer gleich	<code>7.0 >= 3.0</code>	true
<code>==</code>	gleich	<code>6.0 == 5.0</code>	false
<code>!=</code>	ungleich	<code>6.0 != 5.0</code>	true

5.22 Vergleich zweier Fließkommazahlen mit `==` [82]

- Fließkommazahlen sind mit Ungenauigkeit behaftet
- Vergleich mit `==` schlägt häufig wegen Rundungsfehlern fehl
- Daher besser mit Epsilon-Umgebung vergleichen
statt `a == b` besser `(a + epsilon > b) && (a - epsilon < b)`

```
double a = 3.0;
double b = 0.1;
double c = a * b; // -> 0.30000000000000004

// schlägt wegen Rundungsfehler fehl
System.out.println(c == 0.3); // -> false

// besser
System.out.println((c - 0.00001 < 0.3) && (c + 0.00001 > 0.3)); // -> true
```



5.23 Konvertierung von Datentypen [83]

- **Implizite Konvertierung** – kleinere Datentypen werden automatisch in größere Datentypen konvertiert (kein Verlust an Genauigkeit)
 - ▶ passiert automatisch
 - ▶ byte, short, int, long → double, float
 - ▶ byte → short, short → int, int → long
- **Explizite Konvertierung** – Genauigkeit geht verloren, wenn größerer in kleineren Datentyp konvertiert wird
 - ▶ Muss explizit über Cast angefordert werden
 - ▶ double, float → byte, short, int, long
 - ▶ long → int, int → short

Implizite Konvertierung**Explizite Konvertierung**

5.24 Cast-Operator [84]

Cast-Operator führt eine explizite Konvertierung durch

Cast-Operator

- Syntax: (DATENTYP)WERT
- DATENTYP – Datentyp in den konvertiert werden soll
- WERT – Wert, der konvertiert wird

Typkonvertierung



Syntax eines Casts

```
double d = 3.141;
int i = (int) d; // -> 3

long j = 7000000000L;
int k = (int) j; // -> -1589934592
```



Kapitel 6

Verbund (Java Klasse)

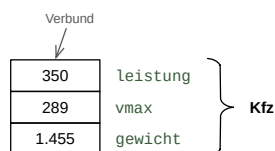
6.1 Motivation [87]

- Viele einzelne Variablen sind verwirrend und schwer zu handhaben
- Häufig gehören Daten zusammen und sollen gemeinsam verarbeitet werden, z. B.
 - Daten eines Studenten (Name, Matrikelnummer etc.)
 - Technische Daten eines Autos (PS, Höchstgeschwindigkeit etc.)
- Man möchte diese Daten zu einer einzigen Variable zusammenfassen

6.2 Verbund/Struktur [88]

- Ein **Verbund** (record) bzw. **Struktur** (struct) erlaubt verschiedene Daten zu einer Variable zusammenfassen
- Er besteht aus Elementen mit unterschiedlichen Datentypen, den **Attributen** (attributes)

Verbund
Struktur
Attributen

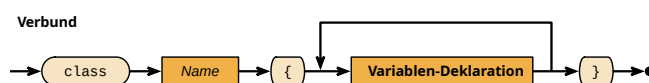


Verbund

6.3 Verbund/Struktur mit Klassen [89]

- In Java wird eine Struktur mit Hilfe einer **Klasse** (class) realisiert (Gründe werden später beleuchtet)
- Eine Klasse setzt sich aus *primitiven Datentypen* oder anderen *Klassen* zusammen
- Syntax: `class NAME { Variablen-Deklaration }`

Klasse



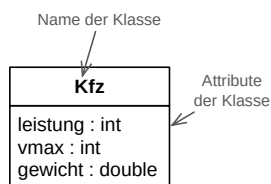
Syntax des Verbundes

6.4 Beispiel: Verbund mit Java Klasse [90]

```
class Kfz {  
  
    /** Leistung in PS */  
    int leistung;  
  
    /** Hoechstgeschwindigkeit in km/h */  
    int vmax;  
  
    /** Leergewicht in Tonnen */  
    double gewicht;  
}
```



6.5 UML-Darstellung einer Klasse [91]



UML-Darstellung der Klasse Kfz

6.6 Speicherung einer Klasse [92]

- In Java müssen Klassen in einer eigenen Datei abgelegt werden
- Datei muss so heißen, wie die Klasse mit der Endung `.java`
`Kfz.java`
- Eclipse beachtet diese Regel automatisch

6.7 Verwenden einer Klasse [93]

- Jede Klasse führt einen neuen Datentyp ein
- Kann als Typ einer Variable benutzt werden
Syntax: `NAME variable;`
- Elemente einer Klasse nennt man **Attribute** (attributes)
- Anders als bei primitiven Typen handelt es sich um Referenztypen (s. u.)

Attribute

```
Kfz bmw;  
Kfz porsche;  
Kfz ferrari;
```



6.8 Referenzdatentypen [94]

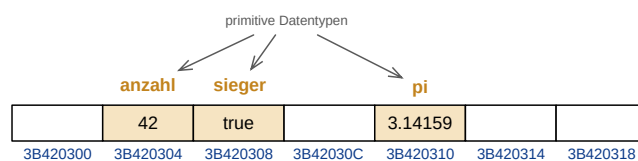
Java unterscheidet zwischen

- **primitiven Datentypen** – Variable repräsentiert Speicherstelle, in der die Daten stehen
 - ▶ `byte`, `short`, `int`, `long`, `boolean`, `char`, `double`, `float`
- **Referenzdatentypen** – Variable repräsentiert Speicherstelle, in der die Adresse der Speicherstelle steht, in der die Daten zu finden sind (Indirektion)
 - ▶ alle anderen (höheren) Datentypen (Klassen, Verbund)
 - ▶ insbesondere Strings und Arrays
- Variable, die auf einen Referenzdatentyp zeigt nennt man **Referenzvariable**

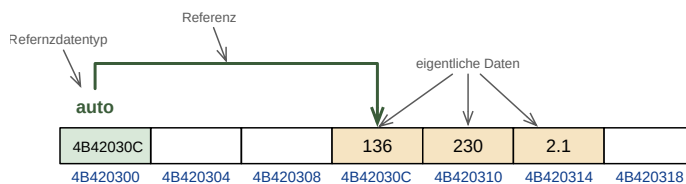
primitiven
Datentypen

Referenzdatentypen

Referenzvariable



```
int anzahl = 42;
boolean sieger = true;
double pi = 3.14159;
```



```
Kfz auto;
auto = new Kfz();
auto.leistung = 136;
auto.vmax = 230;
auto.gewicht = 2.1;
```

Vergleich primitiver Datentyp mit Referenztyp

6.9 Definition von Referenzdatentypen [96]

Definition besteht aus zwei Schritten

1. Anlegen der Variable (Referenzvariable), die die Referenz speichert
Syntax: `DATENTYP variable`;
2. Speicher für die eigentlichen Daten beschaffen
Syntax: `new DATENTYP()`

Häufig werden beide Operationen kombiniert

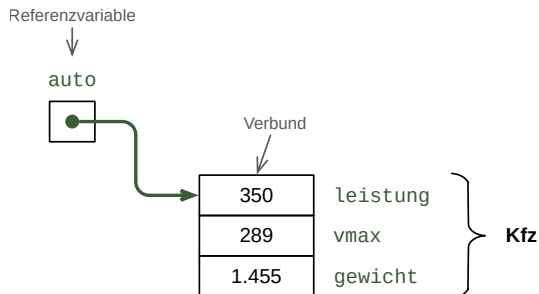
`DATENTYP variable = new DATENTYP();`

```
Kfz auto = new Kfz();
Kfz porsche = new Kfz();
Kfz ferrari = new Kfz();
```



6.10 Referenzdatentypen [97]

```
Kfz auto = new Kfz();
```



Speicherablage eines Verbundes

6.11 Besonderer Wert: null [98]

- Referenzvariablen können einen besonderen Wert haben, der andeutet, dass sie *auf nichts verweisen*. Dieser Wert ist **null**
- Nur zwei mögliche *Zustände* einer Referenzvariable
 - ▶ Sie zeigen auf eine Speicherstelle mit Daten oder
 - ▶ Sie haben den Wert **null**
- Man kann eine Variable mit **null** vergleichen, um zu sehen, ob sie auf Daten zeigt oder nicht

```
Kfz auto = null; // zeigt auf nichts
auto = new Kfz(); // zeigt auf das neue KFZ

System.out.println(auto == null); // -> false
```



6.12 Klasse und Objekt [99]

In Java bezeichnet

- **Klasse** den Datentyp, der neu definiert wird (Vorlage für Objekte)
 - ▶ es gibt immer *nur eine* Klasse
 - ▶ **Kfz** im Beispiel
- **Objekt** Instanzen der Klasse, in der Daten gespeichert werden können
 - ▶ es gibt *beliebig viele* Objekte von einer Klasse
 - ▶ Instanz, auf welche die Referenzen **golf** und **opel** im Beispiel zeigen

Klasse

Objekt



```
Kfz golf = new Kfz();
Kfz opel = new Kfz();
```

6.13 Zugriff auf Elemente [100]

- **Punkt-Operator** (.) erlaubt Zugriff auf die *Attribute* des Verbundes
- Syntax: `variable.ATTRIBUTNAME`
- Wichtig: Variable darf nicht `null` sein

Punkt-Operator

```
Kfz porsche = new Kfz();
porsche.leistung = 350;
porsche.vmax = 289;
porsche.gewicht = 1.455;

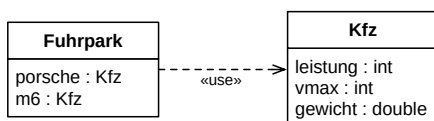
Kfz m6 = new Kfz();
m6.leistung = 507;
m6.vmax = 305;
m6.gewicht = 1.785;
```



6.14 Verschachtelte Definitionen [101]

Klassen können Attribute von Klassen sein

```
class Fuhrpark {
    Kfz porsche = new Kfz();
    Kfz m6 = new Kfz();
}
```

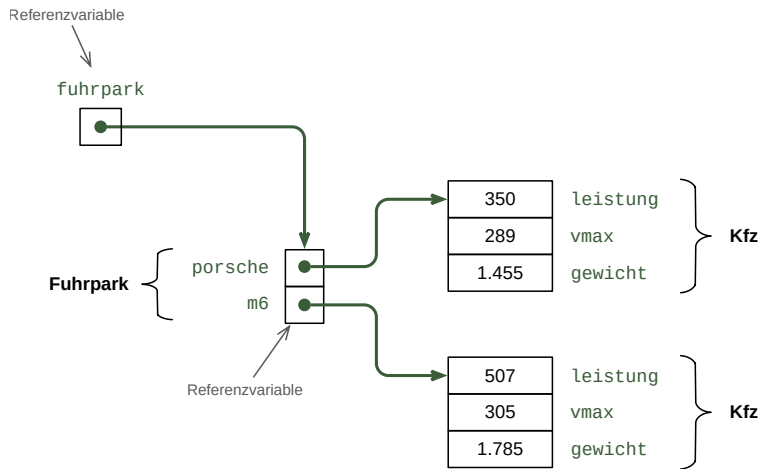


```
Fuhrpark fuhrpark = new Fuhrpark();

fuhrpark.porsche.leistung = 350;
fuhrpark.porsche.vmax = 289;
fuhrpark.porsche.gewicht = 1.455;

fuhrpark.m6.leistung = 507;
fuhrpark.m6.vmax = 305;
fuhrpark.m6.gewicht = 1.785;
```





6.15 Standardwerte [104]

Werden Variablen eines Objektes nicht initialisiert, erhalten sie Standardwerte

- Ganzzahlen: 0
- Fließkommazahlen: 0.0
- Wahrheitswerte: false
- Referenzvariablen: null

```
Kfz auto = new Kfz();

System.out.println(auto.vmax); // -> 0
System.out.println(auto.gewicht); // -> 0.0
```



6.16 Verkettung von Objekten [105]

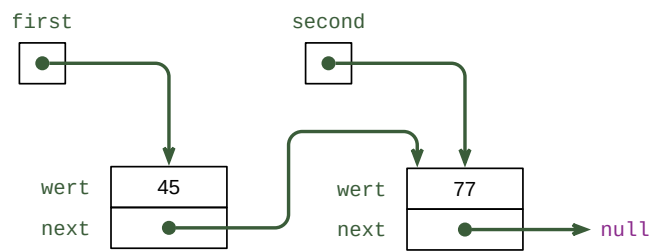
- Objekte können auch wieder Referenzen auf Objekte enthalten
- Man kann so z. B. Ketten von Objekten bilden (**Listen** genannt)
- Mehr dazu in der Vorlesung Algorithmen und Datenstrukturen

Listen

```
class ListElement {
    int wert;
    ListElement next;
}
```



```
ListElement first = new ListElement();  
first.wert = 45;  
  
ListElement second = new ListElement();  
second.wert = 77;  
first.next = second;
```



Verkettete Liste

```
System.out.println(first.wert); // -> 45  
  
System.out.println(first.next.wert); // -> 77
```



Kapitel 7

Arrays

7.1 Typische Probleme [109]

Wie bildet man das Minimum von 100 Werten?

Wie speichert man die Matrikelnummern der 90 Teilnehmer unserer GDI-Vorlesung?

Gemeinsamkeit

- viele unterschiedliche Werte desselben Datentyps
- Anzahl der Werte ist im Voraus bekannt

7.2 Array [110]

Ein *Array* oder *Feld* ist ein zusammenhängender Speicherbereich, der aus N Speicherzellen besteht, die alle den gleichen Datentyp enthalten.

nach G. Büchel, Praktische Informatik

Arrays

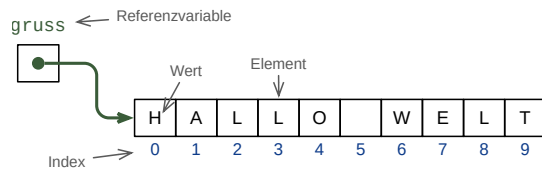
- sind ein **höherer Datentyp**, d. h. sie sind aus einfacheren Datentypen aufgebaut.
- sind Referenzdatentypen
- haben eine feste Größe, die nicht mehr geändert werden kann

höherer Datentyp

Elemente des Arrays

Elemente

- werden über ihre Position (Index) angesprochen
- Index beginnt bei 0



Array als höherer Datentyp

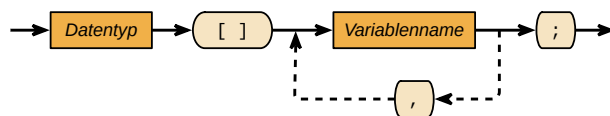
7.3 Deklaration von Arrays [113]

Syntax

- Variablendeklaration: `DATENTYP[] variable;`
- Speicherbeschaffung: `variable = new DATENTYP[GROESSE];`
- Wertzuweisung: `variable[INDEX] = WERT;`
- Wert auslesen: `variable[INDEX]`
- Auslesen der Länge eines Arrays: `variable.length`

Indizes (`INDEX`) beginnen bei 0 und laufen bis `length - 1`

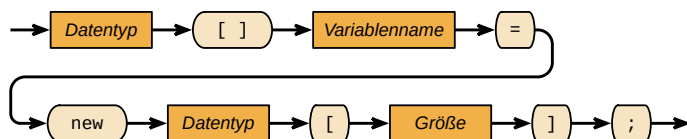
Array-Deklaration



Array-Erzeugung

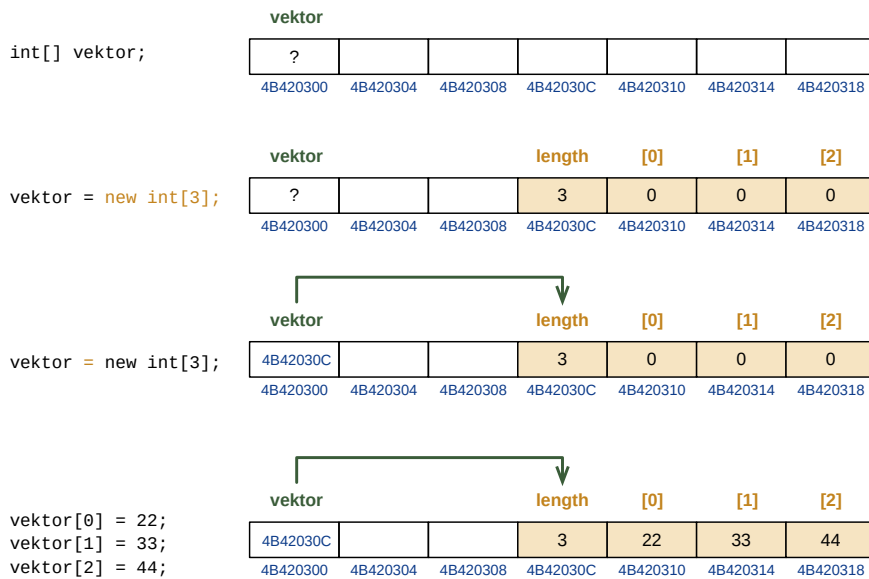


Array-Deklaration und Erzeugung



Syntax für Arrays

7.4 Array im Speicher [115]



Anlegen eines Arrays

7.5 Initialisierung von Arrays [116]

Nach dem Anlegen des Arrays mit `new` haben die Elemente **Standardwerte** (default values)

Standardwerte

- `boolean` → `false`
- `byte`, `short`, `char`, `int`, `long` → `0`
- `double`, `float` → `0.0`
- alle Referenztypen → `null`

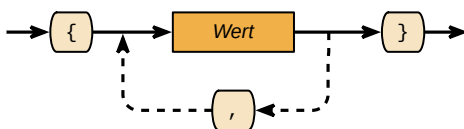
```
int[] array = new int[3];
System.out.println(array[2]); // --> 0
```



- Wenn beim Anlegen bereits alle Elemente bekannt sind, kann man ein Array über ein Literal initialisieren

```
DATENTYP[] variable = { WERT1, WERT2, WERT3 };
```

Array-Literal



Syntax für ein Array-Literal

```
int[] primzahlen = { 2, 3, 5, 7, 11, 13, 17, 19 };
```



```
String[] wochentage = { "Montag", "Dienstag", "Mittwoch",
                        "Donnerstag", "Freitag", "Samstag", "Sonntag" };

int[] leeresArray = {};
```

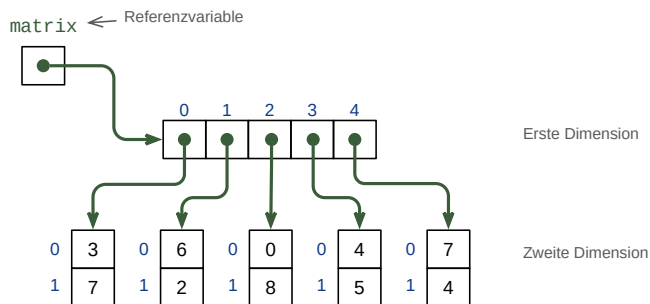
7.6 Mehrdimensionale Arrays [118]

- Arrays können beliebig viele Dimensionen haben, häufig kommen zwei Dimensionen vor
- Mehrdimensionale Arrays werden durch Arrays realisiert, die wieder Referenzen auf Arrays enthalten

```
int[][] matrix = { { 3, 7 }, { 6, 2 }, { 0, 8 }, { 4, 5 }, { 7, 4 } };

System.out.println(matrix.length); // -> 5
System.out.println(matrix[0].length); // -> 2

System.out.println(matrix[1][1]); // -> 2
System.out.println(matrix[4][0]); // -> 7
```



Zweidimensionales Array

Syntax

- Variablendeklaration: `DATENTYP[][] variable;`
- Speicherbeschaffung: `variable = new DATENTYP[GROESSE1][GROESSE2];`
- Wertzuweisung: `variable[INDEX1][INDEX2] = WERT;`
- Wert auslesen: `variable[INDEX1][INDEX2]`
- Auslesen der Länge der ersten Dimension: `variable.length`
- Auslesen der Länge der zweiten Dimension: `variable[INDEX].length`

Kapitel 8

Strings

8.1 Motivation [122]

- Die meisten Programme verarbeiten Zeichenketten
 - ▶ Texte für Anzeigen am Bildschirm
 - ▶ Texte von Eingaben
 - ▶ Daten wie Namen, Adressen, etc.
- Datentyp für Zeichenketten ist daher wichtig

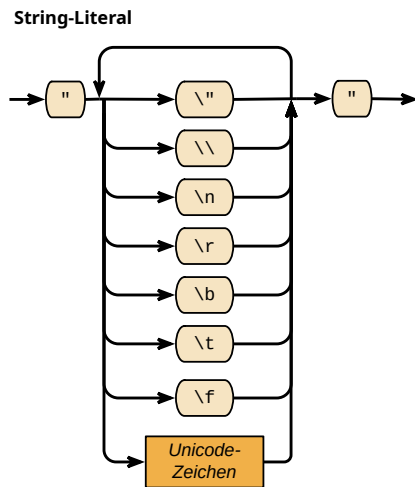
8.2 String [123]

Zeichenketten (**strings**) werden in Java mit dem Datentyp **String** verwaltet

Zeichenketten

- Referenzdatentyp (vgl. Arrays)
- Speichert eine beliebige Anzahl von Zeichen (0, 1, ...)
- Basiert intern auf einem Array von Zeichen (**char**[])
- Kann beliebige Unicode-Zeichen enthalten
- Erzeugung über String-Literal oder **new String()**
- Nach der Erzeugung kann der String nicht mehr verändert werden

8.3 String-Literale [124]



Syntax für String-Literale

String-Literale repräsentieren ein Folge von Zeichen, umschlossen von doppelten Anführungszeichen "

Bestimmte Sonderzeichen müssen speziell notiert (**escaped**) werden (vgl. Zeichenliterale)

escaped

Escape-Sequenz	Bedeutung
\ "	Anführungszeichen ' '
\\	Backslash \
\t	Tabulator
\n	Zeilenvorschub (newline)
\r	Wagenrücklauf (carriage return)
\b	Backspace
\f	Seitenvorschub (form feed)

8.4 String als Referenzdatentyp [126]

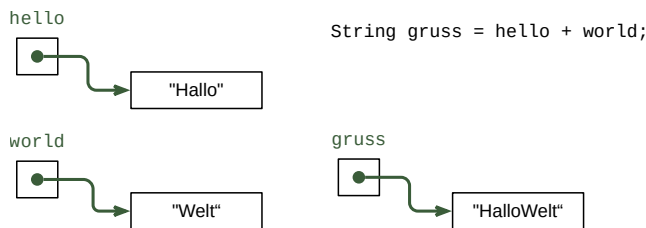
- Deklaration einer *Referenzvariable* für einen String
`String variable;`
- Inhalt der Variable ist nach der Deklaration undefiniert
- Erzeugung eines Strings
 - ▶ über ein Literal
`variable = "INHALT";`
 - ▶ über `new String()`
`variable = new String("INHALT");`
- Kombination aus Zuweisung und Erzeugung
`String variable = "INHALT";`

8.5 Verkettung von Strings [127]

Verkettung (concatenation)

Verkettung

- Strings können mit + zu neuen Strings verkettet werden
- Strings können mit + mit anderen Datentypen verkettet werden
- Ergebnis ist ein neuer String im Speicher
- Auswertung erfolgt strikt von Links nach Rechts



Verkettung von Strings

```
String a = "Hallo" + " " + "Welt";
System.out.println(a); // -> Hallo Welt

String b = "Antwort=" + 42;
System.out.println(b); // -> Antwort=42

String c = "Antwort=" + 39 + 3;
System.out.println(c); // -> Antwort=393

String d = "Antwort=" + (39 + 3);
System.out.println(d); // -> Antwort=42

String e = 39 + 3 + " ist die Antwort";
System.out.println(e); // -> 42 ist die Antwort
```



8.6 Methoden von String [129]

String ist ein *abstrakter Datentyp* und bringt eine Reihe von Methoden mit

- Methoden verändern nie den Inhalt des Strings, sondern erzeugen einen neuen String
- Werden über den **Punkt-Operator** (.) aufgerufen
Syntax: `variable.methode()`

Punkt-Operator

Methode	Rückgabe	Zweck
<code>length()</code>	<code>int</code>	Länge des Strings in Zeichen zurückgeben
<code>charAt(int i)</code>	<code>char</code>	Zeichen an der Stelle <code>i</code> zurückgeben
<code>substring(int start)</code>	<code>String</code>	Teil-String ab der Stelle <code>start</code>



```
String test = "Hallo Java";

// Laenge des Strings
System.out.println(test.length()); // -> 10

// Zeichen an Stelle 6 (Zaehlung beginnt bei 0)
System.out.println(test.charAt(6)); // -> J

// Teilstring ab Stelle 6 (Zaehlung beginnt bei 0)
System.out.println(test.substring(6)); // -> Java
```

8.7 Vergleich von Strings [131]

Strings und Referenzdatentypen vergleicht man *nicht* mit `==`



Strings werden nicht mit `==` verglichen, sondern mit einer speziellen Methode `equals()`

```
String film1 = "Pulp Fiction";
String film2 = "Pulp Fiction";

System.out.println(film1.equals(film2)); // -> true
```



Kapitel 9

Konstanten

9.1 Wozu Konstanten? [133]

- Es gibt fixe Werte, die sich während des Programmablaufs nie ändern
 - ▶ pi, e, N_A
 - ▶ Umsatzsteuersatz
- Diese Werte werden oft mehrmals verwendet
- Man möchte
 - ▶ sie an einer Stelle festlegen, sodass Änderungen zentral möglich sind
 - ▶ ihnen einen sprechenden Namen geben

```
double radius = 10.0;  
double flaeche = 3.14169265 * radius * radius;  
double umfang = 2.0 * 3.14169265 * radius;
```



Es gibt in jedem Programm Werte, die innerhalb des Programms nicht geändert werden, d. h. sie bleiben während des Programmablaufs *konstant*. Einige dieser Werte ändern sich nie, wie z. B. mathematische oder physikalische Konstanten. Andere können sich schon ändern (z. B. Umsatzsteuersatz), die Änderungen sind aber sehr selten und geschehen nicht während der Programmlaufzeit.

Diese konstanten Werte werden normalerweise an mehreren Stellen des Programms verwendet. Setzt man die Zahlenwerte direkt in das Programm ein (so wie im vorliegenden Beispiel), so muss man sie bei einer Änderung auch an mehreren Stellen anpassen. Dabei können leicht Fehler passieren. Im Beispiel ist der Wert von PI falsch angegeben. Entdeckt man den Fehler an einer Stelle, muss man ihn an allen anderen Stellen ebenfalls korrigieren. Hierbei übersieht man schnell eine Stelle.

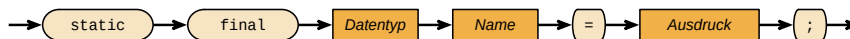
Ein weiterer Vorteil von Konstanten ist, dass man anstatt einer Zahl einen Sprechenden Namen verwenden kann. Wenn im Quelltext nicht 0.19, sondern `UST_SATZ` steht, ist es für einen Leser viel schneller zu erkennen, was gemeint ist.

9.2 Konstanten [134]

- Eine **Konstante** (**constant**) ist ein Wert, der im Programm einmal deklariert wird und sich danach nicht mehr ändern kann
- Konstanten haben einen Datentyp wie Variablen
- Deklaration innerhalb einer Methode
`final TYP NAME = WERT;`
- Deklaration außerhalb einer Methode
`static final TYP NAME = WERT;`

Konstante

Deklaration einer Konstanten



- Namensschema
 - ▶ Großbuchstaben, Zahlen (A–Z, 0–9)
 - ▶ Worttrennung mit Unterstrich (_)
 - ▶ Beispiel: `UST_SATZ`, `MAX_ANZAHL`
- Programm kann beliebig viele Konstanten haben
- Wert der Konstante kann auch berechnet werden

Konstanten verhalten sich ähnlich zu Variablen, d. h. sie haben einen Datentyp und müssen vor der Verwendung deklariert werden. Anders als bei Variablen müssen sie direkt bei der Deklaration mit einem Wert belegt werden und können nachträglich nicht mehr geändert werden.

Die Namenskonvention für Java sieht vor, dass man Konstanten nur mit Großbuchstaben benennt. Da hierdurch keine Camel-Notation mehr möglich ist, trennt man Wörter durch den Unterstrich.

9.3 Beispiel: Innerhalb der Methode [136]

```
class Kreis {  
    public static void main(String[] args) {  
  
        final double PI = 3.14159265;  
  
        double radius = 10.0;  
        double flaeche = PI * radius * radius;  
        double umfang = 2.0 * PI * radius;  
    }  
}
```



9.4 Beispiel: Außerhalb der Methode [137]

```
class Physik {  
  
    static final double PI = 3.14159265;  
    static final double H = 6.62606957e-34;  
    static final double H_QUER = H / (2 * PI);  
  
    public static void main(String[] args) {  
        double frequenz = 120e12;  
        double energie = frequenz * H_QUER;  
    }  
}
```



Das Beispiel zeigt, dass eine Konstante auch berechnet werden kann, wie hier `H_QUER`. Voraussetzung ist, dass die Werte, die in die Berechnung eingehen, selber wieder konstant sind.

Kapitel 10

Auswahanweisungen

10.1 Beispielhafte Probleme [139]

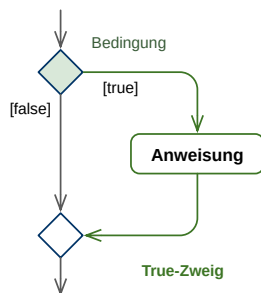
Verhindere, dass der Hamster vor eine Wand läuft.

Öffne das Fenster, wenn die Luft schlecht wird.

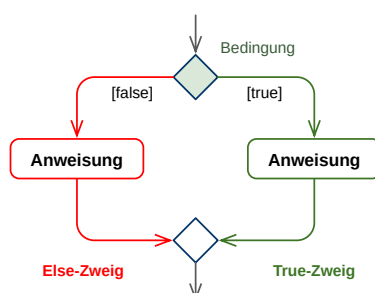
Bestimme das Minimum zweier Zahlen.

10.2 Ein- und Zweiseitige Auswahanweisung [140]

Einseitige Auswahl



Zweiseitige Auswahl



Arten von Auswahanweisungen

■ Einseitige Auswahanweisung

- ▶ Prüfe eine Bedingung, die einen boolschen Wert ergibt
- ▶ Wenn Bedingung **true** ist führe Anweisungen im True-Zweig aus
- ▶ Fahre nach der Auswahanweisung fort

■ Zweiseitige Auswahanweisung

- ▶ Prüfe eine Bedingung, die einen boolschen Wert ergibt
- ▶ Wenn Bedingung **true** ist führe Anweisungen im True-Zweig aus

- ▶ Wenn Bedingung *false* ist führe Anweisungen im Else-Zweig aus
- ▶ Fahre nach der Auswahlanweisung fort

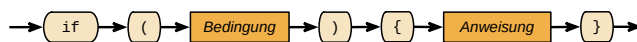
10.3 Einseitige Auswahl in Java (if) [142]

if-Anweisung bietet eine einseitige Auswahl

if-Anweisung

- Syntax: `if (BEDINGUNG){ TRUE-ZWEIG }`
- wenn die logische Bedingung erfüllt ist wird der True-Zweig ausgeführt

if-Anweisung



Syntax der einseitigen if-Anweisung

```
if (antwort == 42) {  
    System.out.println("Die Antwort auf alle Fragen");  
}
```



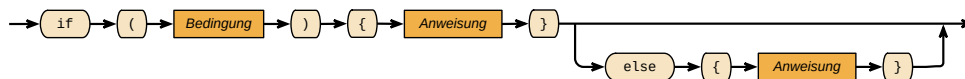
10.4 Zweiseitige Auswahl in Java (if) [143]

if-else-Anweisung bietet eine zweiseitige Auswahl

if-else-Anweisung

- Syntax: `if (BEDINGUNG){ TRUE-ZWEIG } else { ELSE-ZWEIG }`
- wenn die logische Bedingung erfüllt ist wird der True-Zweig ausgeführt, andernfalls der Else-Zweig

if-Anweisung



Syntax der zweiseitigen if-Anweisung

```
if (antwort == 42) {  
    System.out.println("Die Antwort auf alle Fragen");  
} else {  
    System.out.println("Keine Antwort");  
}
```



10.5 Konventionen für Formatierung von if [144]

- Leerzeichen zwischen `if` und `(`
- Leerzeichen zwischen `)` bzw. `else` und `{`
- Kein Leerzeichen zwischen `(` bzw. `)` und Bedingung
- `else` in selbe Zeile wie `}` oder direkt darunter

- True- und False-Zweig werden vier Leerzeichen eingerückt

10.6 Mehrfachverzweigung mit if-else-if [145]

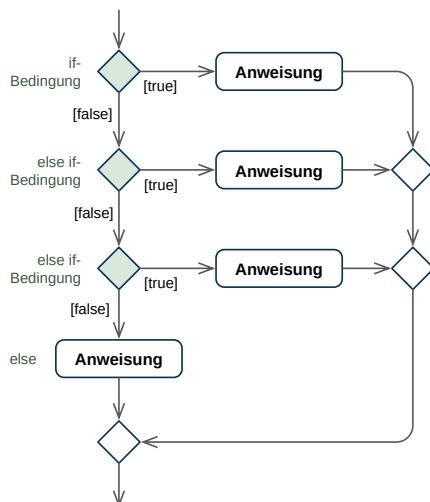
if-else-if-Anweisung bietet eine Mehrfachverzweigung

if-else-if-Anweisung

- Eigentlich nur eine Verschachtelung mehrerer if-else-Anweisungen
- Wird speziell formatiert, um besser lesbar zu sein
- Beliebige viele **else if**, nur ein **else**

Syntax:

```
if (BEDINGUNG1) {
    TRUE-ZWEIG1
} else if (BEDINGUNG2) {
    TRUE-ZWEIG2
} else if (BEDINGUNG3) {
    TRUE-ZWEIG3
} else {
    ELSE-ZWEIG
}
```



Mehrfachauswahl mit if-else-if

```
if (temp < 10) {
    System.out.println("Viel zu kalt");
}
else if (temp < 20) {
    System.out.println("Kalt");
}
else if (temp > 50) {
    System.out.println("Viel zu warm");
}
```



```
}  
else if (temp > 30) {  
    System.out.println("Warm");  
}  
else {  
    System.out.println("Alles super");  
}
```

10.7 Mehrfachverzweigung mit switch [148]

switch-Anweisung bietet andere übersichtlichere Form der Mehrfachverzweigung

switch-Anweisung

Syntax:

```
switch (VARIABLE) {  
    case WERT1:  
        ANWEISUNGEN;  
        break;  
    case WERT2:  
        ANWEISUNGEN;  
        break;  
    ...  
    default:  
        ANWEISUNGEN;  
}
```



Besonderheiten

- **VARIABLE**: muss eine Ganzzahl oder ein String sein
- **WERT**: ein möglicher Wert der Variable als Literal oder Konstante
- **default**: Zweig der ausgeführt wird, wenn kein Wert vorher gepasst hat

```
switch (monat) {  
    case 2: tage = 28; break;  
    case 4: tage = 30; break;  
    case 6: tage = 30; break;  
    case 9: tage = 30; break;  
    case 11: tage = 30; break;  
    default: tage = 31;  
}
```



Die Variable, die im switch-Statement verwendet wird (hier `monat`) muss entweder zuweisungs-kompatibel mit dem Typ `int` sein (`byte`, `short`, `char`, `int`) oder es muss sich um eine Referenz auf eine Enumeration oder einen String handeln (Enumerations werden später eingeführt). Es ist nicht möglich im switch-Statement Variablen von anderen Typen zu verwenden, d. h.

`double`, `float`, `boolean` etc. sind nicht verwendbar. Der Grund hierfür liegt in der internen Implementierung von `switch` in der Java Virtual Machine.

Die Werte in den `case`-Ästen müssen Konstanten vom Typ `int`, `String` oder einer Enumeration sein. Sie brauchen keinen expliziten Block zu schreiben, d. h. anders als beim `if` bezieht sich der `case`-Ast auf alle Statements bis zum nächsten `case` oder `break`.

Der `default`-Ast wird ausgeführt, wenn keine der `Case`-Konstanten gepasst hat.

Bei Java fällt bei einem `switch`-Statement der Kontrollfluss durch alle `case`-Statements, die auf das `case` folgen, bei dem die Bedingung zutrifft. Daher muss man immer explizit ein `'break'` hinter die `case`-Statements schreiben, wenn man dieses Verhalten nicht möchte – was fast immer der Fall ist. Java hat dieses Verhalten von C geerbt, wo es ebenfalls als obskur gilt.

Da bei einem `case` der Kontrollfluss bis zum ersten `break` weiterläuft, kann man das obige Beispiel auch noch kompakter schreiben als:

```
switch (monat) {  
    case 2:  
        tage = 28; break;  
    case 4:  
    case 6:  
    case 9:  
    case 11:  
        tage = 30; break;  
    default:  
        tage = 31;  
}
```



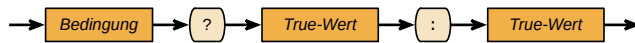
10.8 Konventionen für Formatierung von switch [150]

- Leerzeichen zwischen `switch` und `(`
- Leerzeichen zwischen `)` und `{`
- Kein Leerzeichen zwischen `(` bzw. `)` bei Variable
- `case` jeweils um vier Leerzeichen eingerückt
- Anweisungen hinter `:` von `case` oder darunter, acht Leerzeichen eingerückt

10.9 Fragezeichen-Operator [151]

- Bedingungen mit `if` und `case` können nicht in Ausdrücken eingesetzt werden
- Für die Verwendung in Ausdrücken gibt es einen speziellen **Bedingungsoperator (Fragezeichen-Operator)**
- Einziger Operator in Java mit *drei* Operanden (**ternärer Operator**)
- Syntax: `BEDINGUNG ? TRUE-WERT : FALSE-WERT`

Bedingungsoperator
Fragezeichen-Operator
ternärer Operator

Fragezeichen-Operator

Syntax des ternären Operators

10.10 Beispiel für Fragezeichenoperator: Absolutbetrag [152]

Mit if

```
if (a < 0) {  
    b = -a;  
}  
else {  
    b = a;  
}
```



Mit Fragezeichen-Operator

```
b = a < 0 ? -a : a;
```



Kapitel 11

Schleifen

11.1 Beispielhafte Probleme [154]

Bilde die Summe der Quadrate der Zahlen zwischen 1 und n.

Berechne den größten gemeinsamen Teiler mit dem Euklidischen Algorithmus.

Lies Zahlen von der Tastatur, bis der Benutzer einen negativen Wert eingibt.

11.2 Schleife [155]

Eine *Schleife* (loop) ist ein Block von Anweisungen, der solange mehrfach nacheinander ausgeführt wird, wie ein *Vergleichsausdruck*, mit dem die Schleife kontrolliert wird, wahr ist.

G. Büchel, Praktische Informatik

11.3 Arten von Schleifen [156]

Ein Schleife heißt *kopfgesteuert*, wenn jeweils vor dem Schleifendurchlauf kontrolliert wird, ob die logische Bedingung der Schleife immer noch gültig ist. Ist die Bedingung bereits von Anfang an falsch, wird sie nicht durchlaufen (*abweisend*).

nach G. Büchel, Praktische Informatik

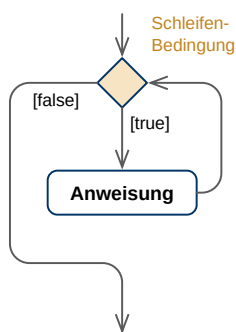
1. Werte den booleschen Ausdruck (Schleifenbedingung) aus
2. Falls die Schleifenbedingung den Wert *false* liefert, beende die Schleife und fahre danach fort
3. Falls die Schleifenbedingung den Wert *true* liefert, führe die Anweisung (Iterationsanweisung) aus und gehe danach zu 1.

Ein Schleife heißt *fußgesteuert*, wenn jeweils erst am Ende eines Schleifendurchlaufes kontrolliert wird, ob die logische Bedingung der Schleife noch gültig ist. Sie wird mindestens einmal durchlaufen (*nicht abweisend*).

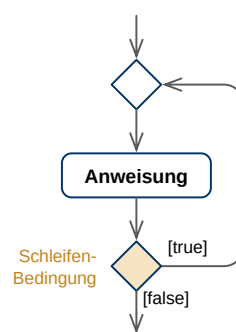
nach G. Büchel, Praktische Informatik

1. Führe die Anweisung (Iterationsanweisung) aus
2. Werte den booleschen Ausdruck (Schleifenbedingung) aus
3. Falls die Schleifenbedingung den Wert *false* liefert, beende die Schleife und fahre danach fort
4. Falls die Schleifenbedingung den Wert *true* liefert, gehe zu 1.

Kopfgesteuerte Schleife



Fußgesteuerte Schleife



Arten von Schleifen

11.4 while-Schleife [159]

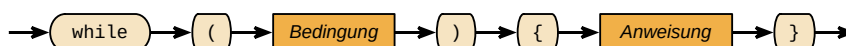
- **while-Schleife** (**while loop**) prüft *vor* jedem Durchlauf eine boolesche Bedingung (Vergleichsausdruck)
- *kopfgesteuert* und *abweisend*
- Syntax: `while (BEDINGUNG){ ANWEISUNGEN }`

while-Schleife

```
while (antwort < 42) {
    antwort++;
}
```



while-Schleife



Syntax der while-schleife

Der Rumpf der while-Schleife wird nur betreten, wenn die Bedingung des booleschen Ausdrucks wahr ist. Wenn man eine Schleife wünscht, die mindestens einmal durchlaufen wird, muss man die do/while-Schleife wählen. Die Variable auf die getestet wird sollte richtig in-

initialisiert sein und man sollte nicht vergessen sie im Schleifenrumpf zu verändern, da man andernfalls in einer Endlosschleife landet.

Wenn die Anzahl der Schleifendurchläufe schon im Voraus feststeht, sollten Sie die for-Schleife verwenden. Das hier gewählte Beispiel ist also nicht optimal, da man es besser mit for implementiert hätte. While ist gut geeignet für Schleifen, bei denen während des Durchlaufens erst festgestellt werden kann wann der Abbruch erfolgen soll, z. B. bei linearen Suchen etc.

11.5 Beispiel: while-Schleife [160]

```
// Beispiel: Gib alle Zahlen von 1 bis 10 aus
int i = 1;
while (i <= 10) {
    System.out.println(i);
    i++;
}
```



```
// Beispiel: Berechne Summe der Zahlen von 1 bis 10
int i = 1;
int summe = 0;
while (i <= 10) {
    summe += i;
    i++;
}
System.out.println(summe);
```



11.6 do-while-Schleife [161]

do-while-Schleife (do-loop) ist *fußgesteuert* (*nicht abweisend*)

do-while-Schleife

- Syntax: `do { ANWEISUNGEN } while (BEDINGUNG);`

```
do {
    n = readNatuerlicheZahl();
} while (n >= 0);
```



do-while-Schleife



Syntax der do-while-schleife

Der Rumpf der do/while-Schleife wird immer mindestens einmal betreten, da der Test erst am Ende erfolgt. Wenn man eine Schleife wünscht, die die Bedingung vorher testet, muss man die

while-Schleife verwenden. Die Variable auf die getestet wird sollte richtig initialisiert sein und man sollte nicht vergessen sie im Schleifenrumpf zu verändern, da man andernfalls in einer Endlosschleife landet.

Wenn die Anzahl der Schleifendurchläufe schon im Voraus feststeht, sollten Sie die for-Schleife verwenden. Das hier gewählte Beispiel ist also nicht optimal, da man es besser mit for implementiert hätte.

11.7 Beispiel: do-while-Schleife [162]

```
// Beispiel: Gib alle Zahlen von 1 bis 10 aus
int i = 1;
do {
    System.out.println(i);
    i++;
} while (i <= 10);
```



```
// Beispiel: Berechne Summe der Zahlen von 1 bis 10
int i = 1;
int summe = 0;
do {
    summe += i;
    i++;
} while (i <= 10);

System.out.println(summe);
```



11.8 for-Schleife [163]

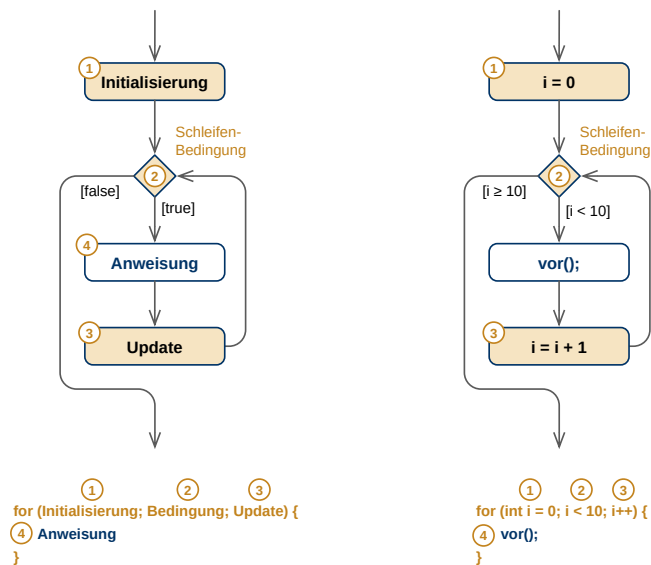
for-Schleife (for-loop) ist *kopfgesteuerte (abweisende Schleife)*

for-Schleife

- bietet mehr Möglichkeiten als die while-Schleife
- Syntax: `for (INIT; BEDINGUNG; UPDATE){ ANWEISUNGEN }`
 - ▶ **INIT**: initialisiert Variablen (optional)
 - ▶ **BEDINGUNG**: Schleife läuft, solange die Bedingung wahr ist
 - ▶ **UPDATE**: Operationen, die nach jedem Durchlauf durchgeführt werden

```
for (int i = 0; i < 10; i++) {
    vor();
}
```





Aktivitätsdiagramm einer for-Schleife

Wie bei `if` kann man auch bei `for` den Block weglassen, wodurch sich die `for`-Schleife auf das nächste Statement bezieht. Hiervon ist aber, genauso wie bei `if`, dringend abzuraten, da es den Source-Code erheblich schlechter lesbar macht.

Man kann in der `for`-Schleife auch einzelne Teile weglassen. Daher sind die folgenden Schleifen korrekter Java-Code:

```

int i = 0;
for (; i < 10; i++) {
    System.out.println("Variable außerhalb deklariert");
}
  
```



```

int j = 0;
for (; j < 10;) {
    j++;
    System.out.println("Variable ausserhalb deklariert");
    System.out.println("Inkrement innerhalb");
}
  
```



```

for (;;) {
    System.out.println("Endlosschleife");
}
  
```



Da sie aber verwirrend sind, sollte man solche Formen normalerweise nicht verwenden.

Es ist zusätzlich möglich, in der for-Schleife mehrere Initialisierungen und Veränderungen parallel durchzuführen. Hierzu dient das Komma (,) als Trenner. Aber auch dieses Konstrukt ist aber mit Vorsicht zu genießen:

```
for (i = 0, j = 0; j < 10; i++, j++) { }
```



11.9 Beispiel: for-Schleife [165]

```
// Beispiel: Gib alle Zahlen von 1 bis 10 aus
for (int i = 1; i <= 10; i++) {
    System.out.println(i);
}
```



```
// Beispiel: Berechne Summe aller Zahlen von 1 bis 10
int summe = 0;

for (int i = 1; i <= 10; i++) {
    summe += i;
}

System.out.println(summe);
```



11.10 foreach-Schleife [166]

Iterieren über Arrays (von Anfang bis Ende) eine sehr häufige Aufgabe in der Programmierung

```
// Beispiel: Berechne die Durchschnittsnote mit normalem for
double[] noten = { 1.0, 2.0, 1.3, 2.3, 4.0, 2.3 };

double summe = 0.0;

for (int i = 0; i < noten.length; i++) {
    double note = noten[i];
    summe += note;
}

double durchschnitt = summe / noten.length;

System.out.println(durchschnitt); // -> 2.15
```



- **foreach-Schleife** bietet verkürzte Schreibweise für das Durchlaufen eines Arrays von Anfang bis Ende

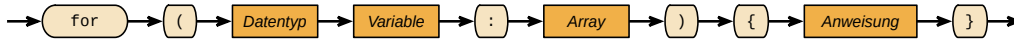
foreach-Schleife

- Deklariert eine **Laufvariable** vom Typ der Array-Elemente, die das Array Schrittweise durchläuft
- Syntax:

```
for (DATENTYP VARIABLE : ARRAY){ ANWEISUNGEN }
```

Laufvariable

foreach-Schleife



Syntax der foreach-Schleife

```
// Beispiel: Berechne die Durchschnittsnote mit foreach
double[] noten = { 1.0, 2.0, 1.3, 2.3, 4.0, 2.3 };

double summe = 0.0;

for (double note : noten) {
    summe += note;
}

double durchschnitt = summe / noten.length;

System.out.println(durchschnitt); // -> 2.15
```



11.11 Schleifenabbruch [169]

Aktueller Schleifendurchlauf kann abgebrochen werden

- **break;** – Schleife wird ganz verlassen
(Sprung an die Anweisung nach der Schleife)
- **continue;** – Beginnt sofort einen neuen Schleifendurchlauf
(Sprung in die Prüfung der Bedingung)

break;**continue;**

Wird häufig als schlechter Programmierstil angesehen. Alternativen

- boolesche Kontrollvariable als Ersatz für **break**
- zusätzliches **if** als Ersatz für **continue**

```
// Beispiel: Suche kleinste Zahl, die durch 8 und 14 teilbar ist (mit break)
for (int i = 1; i <= 8*14; i++) {
    if ((i % 8 == 0) && (i % 14 == 0)) {
        System.out.println(i);
        break;
    }
}
```



```
// Beispiel: Suche kleinste Zahl, die durch 8 und 14 teilbar ist (ohne break)
```



```
boolean gefunden = false;
for (int i = 1; i <= 8*14 && !gefunden; i++) {
    if ((i % 8 == 0) && (i % 14 == 0)) {
        System.out.println(i);
        gefunden = true;
    }
}
```

11.12 Codekonventionen [171]

- Hinter `do`, `for` und `while` ein Leerzeichen
- Vor und hinter den Operatoren ein Leerzeichen
- falls Blockanweisung:
 - ▶ `)` und `{` in dieselbe Zeile, durch Leerzeichen getrennt
 - ▶ `}` unter `i` von `if` bzw. `w` von `while`
- innere Anweisungen um 4 Spalten einrücken
- geschachtelte Schleifen vermeiden (→ Prozeduren)

Kapitel 12

Methoden (Prozeduren)

12.1 Unterprogramme und Funktionen [173]

Programme werden schnell sehr groß

- Unübersichtlich
- Wiederholung identischer oder ähnlicher Algorithmen
 - ▶ sortieren einer Liste
 - ▶ suchen eines Elements

Lösung

- Programm wird in kleinere Einheiten zerlegt (*Unterprogramme*)
- Unterprogramme werden aufgerufen
- Unterprogramme können sich gegenseitig aufrufen

12.2 Begriffe [174]

In Java nennt man ein Unterprogramm grundsätzlich **Methode** (method)

Methode

Weitere Begriffe

- **Funktion** (function) gibt einen Wert zurück
- **Prozedur** (procedure) gibt nichts zurück

Funktion

Prozedur

12.3 Methodendeklaration [175]

- **Methodendeklaration** führt eine neue Methode ein
- Über **Parameter** können der Methode Daten übergeben werden
- Methode kann *beliebig viele Parameter* haben
- spezieller Rückgabetyt **void** für Methoden, die nichts zurückliefern

Methodendeklaration

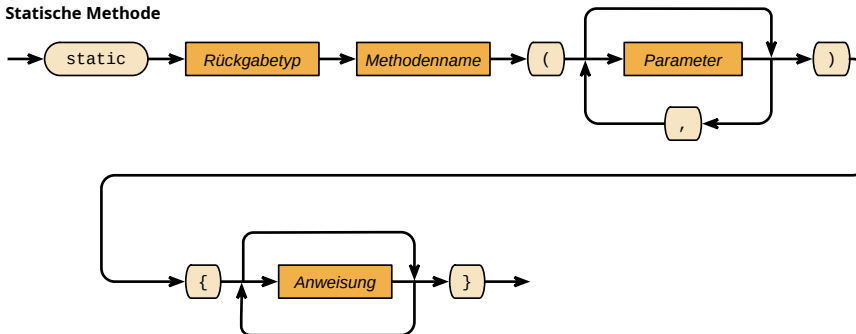
Parameter

```
static Rückgabetyt Name (Parameter) { Anweisungen }
```



```
static void printSum(int a, int b) {
    int sum = a + b;
    System.out.println(sum);
}
```

Statische Methode



Parameter



Syntax einer Methodendeklaration

12.4 Arten von Methoden [177]

- Methoden ohne Rückgabe und ohne Parameter
`static void ohneAlles(){ ... }`
- Methoden ohne Rückgabe und mit Parameter
`static void mitParameter(int a){ ... }`
- Methode mit Rückgabe und ohne Parameter
`static int mitRueckgabe(){ ... }`
- Methode mit Rückgabe und mit Parameter
`static int mitBeidem(int a, int b){ ... }`

12.5 Signatur [178]

- Der Compiler erkennt Methoden anhand ihrer **Signatur** (type signature)
- Signatur besteht aus
 - ▶ *Name* der Methode
 - ▶ *Typen der Parameter* einer Methode (nicht den Namen der Parameter)
- Der Rückgabetyp ist *nicht* Teil der Signatur
- Beispiele
 - ▶ `int max(int a, int b)` → Signatur: `max(int, int)`
 - ▶ `int max(int x, int y)` → Signatur: `max(int, int)`
 - ▶ `void max(int a, int b)` → Signatur: `max(int, int)`

Signatur

► `float min(float a, float b)` → Signatur: `min(float, float)`

12.6 Return-Anweisung [179]

Mit dem **Return-Anweisung** (return statement) wird ein Wert von der Methode zurückgegeben

[Return-Anweisung](#)

- Syntax: `return AUSDRUCK;`
- Typ des Ausdrucks muss zum Rückgabebetyp der Methode passen
- `return` beendet die Methode, nachfolgende Anweisungen werden nicht ausgeführt

return-Anweisung



Syntax der Return-Anweisung

```
static int addiere(int a, int b) {  
    int summe = a + b;  
    return summe;  
}
```



```
static int addiere(int a, int b) {  
    return a + b;  
}
```



```
static double mittelwert(double a, double b, double c, double d) {  
    return (a + b + c + d) / 4.0;  
}
```



12.7 Regeln für die Return-Anweisung [181]

Methode mit Rückgabebetyp `void`

- wird beendet, wenn Ablauf am Ende der Methode angekommen ist
- *kann* durch ein `return` auch an anderer Stelle beendet werden

Methode mit anderem Rückgabebetyp

- *muss* mit `return` einen Wert zurückgeben
- Compiler überprüft, ob auf allen möglichen Wegen ein `return` erfolgt

Mehrere `return` in einer Methode können für Anfänger verwirrend sein, gilt daher oft als schlechter Programmierstil: **single entry, single exit**




```
static void printMax(int a, int b) { // ohne return
    if (a > b) {
        System.out.println(a);
    }
    else {
        System.out.println(b);
    }
}
```

```
static void printMax(int a, int b) { // mit return
    if (a > b) {
        System.out.println(a);
        return;
    }

    System.out.println(b);
}
```



Mehrere return-Anweisungen

```
static int max(int a, int b) {
    if (a > b) {
        return a;
    }
    else {
        return b;
    }
}
```



Nur eine return-Anweisung

```
static int max(int a, int b) {
    int ergebnis;

    if (a > b) {
        ergebnis = a;
    }
    else {
        ergebnis = b;
    }

    return ergebnis;
}
```



12.8 Methodenaufruf [185]

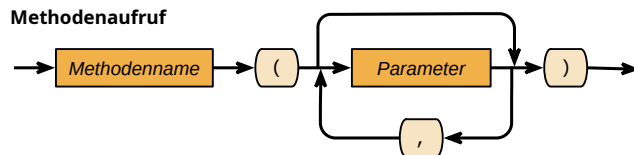
Methoden können über ihren Namen aufgerufen werden

Syntax: `METHODENNAME(PARAMETER);`

- **Formalparameter** Parameter bei der Deklaration (*Variablen*)
- **Aktualparameter** aktueller Wert beim Aufruf (*Wert*)
 - ▶ Anzahl entspricht Formalparametern
 - ▶ Typ ist kompatibel mit Formalparametern

Formalparameter

Aktualparameter



Syntax Methodenaufruf

```
static double mittelwert(double a, double b, double c, double d) {
    return (a + b + c + d) / 4.0;
}

public static void main(String[] args) {
    double durchschnitt = mittelwert(1.0, 2.7, 1.3, 4.0);
    System.out.println(durchschnitt); // -> 2.25
}
```



Formalparameter: `a, b, c, d`

Aktualparameter: `1.0, 2.7, 1.3, 4.0`

12.9 Pass-by-Value beim Methodenaufruf [187]

Übergabe von Daten an Methode erfolgt per **Wertübergabe** (pass by value) (im Gegensatz zu **Referenzübergabe** (pass by reference))

Wertübergabe

Referenzübergabe

- Wert des Aktualparameters wird kopiert
- Methode arbeitet auf einer Kopie
- Wert beim Aufrufer bleibt unverändert

Dies gilt auch für Referenzvariablen

```
static void methode(int a) {
    a++; // schlechter Stil (s.u.)
    System.out.println("a = " + a);
}

public static void main(String[] args) {
    int b = 17;
}
```



```
methode(b);
System.out.println("b = " + b);
}
```

Ausgabe

```
a = 18
b = 17
```



```
static void methode(int[] a) {
    a[0]++;
}

public static void main(String[] args) {
    int[] b = { 1, 2, 3 };
    System.out.println("b[0] = " + b); // vor Aufruf

    methode(b);
    System.out.println("b[0] = " + b); // nach Aufruf
}
```

**Ausgabe**

```
b[0] = 1
b[0] = 2
```

**12.10 Dokumentation von Methoden [192]**

Methoden werden mit **JavaDoc** dokumentiert (→ Pflichtübungen)

JavaDoc

- Spezieller Kommentar mit `/** */`
- Parameter werden über `@param` erläutert
- Rückgabewert wird mit `@return` beschrieben

```
/**
 * Summiert zwei Zahlen.
 *
 * @param a erste Zahl
 * @param b zweite Zahl
 * @return Summe der zwei Zahlen
 */
static int sum(int a, int b) {
    return a + b;
}
```



12.11 Überladene Methoden [193]

Überladene Methode (overloaded method)

Überladene Methode

- In Java dürfen mehrere Methoden denselben Namen haben, solange sich die Parameter unterscheiden
 - in der Anzahl
 - in dem Typ
 - in Anzahl und Typ
- Müssen also unterschiedliche *Signaturen* haben
- Compiler wählt beim Aufruf die Methode mit den richtigen Parametern aus

```
static int max(int a, int b) { // Variante 1
    return a > b ? a : b;
}

static int max(int a, int b, int c) { // Variante 2
    return max(a, max(b, c));
}

static double max(double a, double b) { // Variante 3
    return a > b ? a : b;
}

static double max(double a, double b, double c) { // Variante 4
    return max(a, max(b, c));
}
```



```
public static void main(String[] args) {

    System.out.println(max(1, 2)); // Aufruf Variante 1

    System.out.println(max(3, 2, 5)); // Aufruf Variante 2

    System.out.println(max(1.0, 2.0)); // Aufruf Variante 3

    System.out.println(max(3.0, 2.0, 5.0)); // Aufruf Variante 4
}
```



12.12 Vararg-Methoden [196]

Java erlaubt Methoden mit beliebiger Anzahl von Parametern: **Vararg-Methoden**

Vararg-Methoden

- Methode kann beliebig viele normale Parameter haben (0–n)
- Letzter Parameter kann Vararg-Parameter sein (symbolisiert durch Ellipse)

- Syntax: `TYP... name`
- Parameter stehen als Array vom gegebenen Typ zur Verfügung

```
static int max(int... werte) { /* ... */ }
static int summe(int a, int b, int... summanden) { /* ... */ }
static void print(String text, double... zahlen) { /* ... */ }
```



```
static int max(int... werte) {
    int ergebnis = 0;

    for (int wert : werte) {
        ergebnis = wert > ergebnis ? wert : ergebnis;
    }

    return ergebnis;
}

public static void main(String[] args) {
    int maximum = max(190, 230, 155, 891, 452);

    System.out.println(maximum); // -> 891

    System.out.println(max(190)); // -> 190

    System.out.println(max()); // -> 0
}
```



12.13 Lokale Variablen [198]

Methoden können eigene Variablen haben, die **lokalen Variablen** (local variable)

lokalen Variablen

- alle im Methodenrumpf deklarierten Variablen sind lokal
- alle Parameter verhalten sich wie lokale Variablen
- sind nur im Rumpf der Methode sichtbar
- gehen nach Beenden der Methode verloren
- Wert kann über `return` an Aufrufer zurückgegeben werden

Variablen der aufrufenden Methode

- in der aufgerufenen Methode nicht sichtbar
- Wertübergabe an aufgerufene Methode über Parameter

```
static int sum(int a, int b) {
    // Variablen x, y und summe nicht sichtbar
    int ergebnis; // lokale Variable
}
```



```
    ergebnis = a + b;
    return ergebnis; // Rückgabe des Wertes der Variable
}

public static void main(String[] args) {
    int x = 5; // lokale Variable
    int y = 10; // lokale Variable
    int summe; // lokale Variable
    summe = sum(x, y);
    System.out.println(summe); // -> 15
}
```

12.14 Sichtbarkeit von Variablen [200]

Sichtbarkeit (visibility): Variablen sind nur an bestimmten Stellen sichtbar

Sichtbarkeit

- Lokale Variablen im Rumpf der Methode
sichtbar von Deklaration bis Methodenende
- Lokale Variablen in einem Block
sichtbar von Deklaration bis Ende des Blocks
- Variable im Initialisierungsteil einer for-Schleife
sichtbar im Rumpf der Schleife
- Parameter einer Methode
sichtbar von Deklaration bis Methodenende

```
static int maximum(int[] werte) {
    int laenge = werte.length;
    int max = 0;

    for (int i = 0; i < laenge; i++) {
        if (werte[i] > max) {
            max = werte[i];
        }
    }

    return max;
}

public static void main(String[] args) {
    int[] werte = { 8, 17, 5, 42, 3, 41, 8 };
    System.out.println(maximum(werte)); // -> 42
}
```



```

static int maximum( int[] werte) {
    int laenge = werte.length;
    int max = 0;

    for( int i = 0; i < laenge; i++) {
        if(werte[i] > max) {
            max = werte[i];
        }
    }

    return max;
}

```

```

public static void main(String[] args) {
    int[] werte = { 8, 17, 5, 42, 3, 41, 8 };
    System.out.println(maximum(werte)); // -> 42
}

```

Sichtbarkeitsbereich von Variablen

12.15 Parameter als Variablen [203]

Methodenparameter verhalten sich wie lokale Variablen: Zuweisung an Parameter ist möglich aber *schlechter Stil*

```

// Pfui!
static int max(int a, int b) {
    a = a > b ? a : b;
    return a;
}

```



```

// Richtig
static int max(int a, int b) {
    int ergebnis = a > b ? a : b;
    return ergebnis;
}

```



12.16 Globale Variablen [204]

Globale Variablen (global variables) werden außerhalb von Methoden deklariert und sind in allen Methoden sichtbar

Globale Variablen

- Syntax: `static Datentyp name;`
- Sollten vermieden werden, da Programme mit globalen Variablen schwerer zu verstehen sind

```

class Global {

```



```
static int zaehler;

static void methode() {
    zaehler++;
}

public static void main(String[] args) {
    methode();
    methode();
    System.out.println(zaehler); // -> 2
}
```

12.17 Seiteneffekt [207]

Seiteneffekt (side effect) bezeichnet die Veränderung von globalen Zustand aus einer Methode

Seiteneffekt

■ Nachteile

- ▶ in Signatur der Methode nicht erkennbar
- ▶ erschwert Fehlersuche
- ▶ schränkt Wiederverwendbarkeit ein
- ▶ schlechter Programmierstil

■ Guter Programmierstil

- ▶ keine globalen Variablen verwenden
- ▶ Werteübergabe an Methode ausschließlich per Parameter
- ▶ Ergebnis über Rückgabewert zurückgeben

Kapitel 13

Rekursion

13.1 Idee der Rekursion [209]

Um Rekursion zu verstehen, muss man zuerst Rekursion verstehen.

Rekursion (**recursion**) Algorithmus, bei dem sich eine Methode selbst aufruft

Rekursion

- Erlaubt elegante Lösung vieler Probleme, die sich iterativ nur kompliziert lösen lassen
- Kann immer auch in einen iterativen Algorithmus umgewandelt werden

13.2 Beispiel: Fakultätsberechnung [210]

Rekursiv

```
static long fakRek(int n) {  
    return (n > 1) ? n * fakRek(n - 1) : 1;  
}
```



Iterativ

```
static long fakIter(int n) {  
    long ergebnis = 1;  
  
    for (int i = 2; i <= n; i++) {  
        ergebnis *= i;  
    }  
  
    return ergebnis;  
}
```



13.3 Beispiel: Fibonacci-Zahlen [211]

$$f(n) = \begin{cases} 1 & n = 1 \\ f(n-1) + f(n-2) & \text{andernfalls} \end{cases}$$

```
/**
 * Rekursive Berechnung der Fibonacci-Zahl zu n.
 * @param n Wert, für den die Fibonacci-Zahl berechnet werden soll
 * @return Fibonacci Zahl zu n
 */
static long fibRek(int n) {
    return (n > 2) ? fibRek(n - 1) + fibRek(n - 2) : 1;
}
```



```
/**
 * Iterative Berechnung der Fibonacci-Zahl zu n.
 * @param n Wert, für den die Fibonacci-Zahl berechnet werden soll
 * @return Fibonacci Zahl zu n
 */
static long fibIter(int n) {
    long n2 = 1;
    long n1 = 1;
    long n0 = 1;

    for (int i = 3; i <= n; i++) {
        n0 = n1 + n2;
        n2 = n1;
        n1 = n0;
    }

    return n0;
}
```



Kapitel 14

Speicherverwaltung

14.1 Grundbegriffe [214]

Zwei grundlegende Arten von Speicher

- **Stack** auch genannt
 - ▶ **Kellerspeicher**
 - ▶ **Stapelspeicher**
 - ▶ **LIFO-Speicher** (last in first out)
- **Heap** zu Deutsch Haufen und Halde

Stack

Kellerspeicher

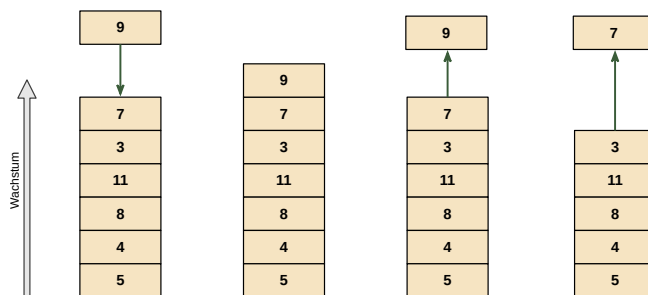
Stapelspeicher

LIFO-Speicher

Heap

14.2 Datenstruktur Stack [215]

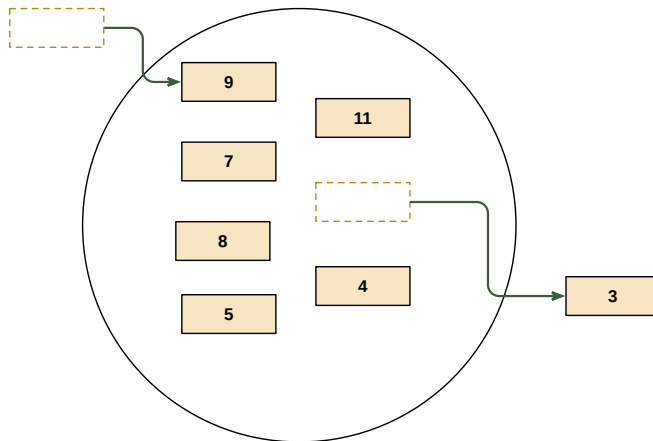
- Zugriff nur auf das oberste Element
- Last in First Out



Aufbau eines Stacks

14.3 Datenstruktur Heap [216]

Wahlfreier Zugriff auf die Elemente



Aufbau eines Heaps

14.4 Einsatz des Stacks [217]

Grundlegende Abläufe

- Methoden können Methoden rufen
- Lokale Variablen leben immer bis zum Ende der Methode und müssen nach dem Ende der Methode zerstört werden
- Methoden übergeben Parameter an andere Methoden
- Methoden geben Werte zurück

Wie verwaltet man den Speicher für Variablen, Parameter und Rückgabewerte?

14.5 Methodenaufruf und Stack [218]

Aufruf einer Methode

- **Aktivierungssatz** (stack frame) wird auf dem Stack erzeugt, enthält
 - ▶ Parameter
 - ▶ lokale Variablen
 - ▶ Platz für interne Rechenoperationen der Methode

Aktivierungssatz

Rückkehr der Methode

- Rückgabewert wird auf den Stack des Aufrufers geschrieben
- Aktivierungssatz der Methode wird vom Stack entfernt
 - ▶ Speicher von lokale Variablen werden gelöscht
 - ▶ Speicher für Parameter wird freigegeben

Da es sich bei der Java-VM nicht um eine Register- sondern eine Stack-Maschine handelt, werden alle Rechenoperationen des Java-Programms auf einem sogenannten **Operanden-Stack** (operand stack) durchgeführt. Dieser wird ebenfalls im Aktivierungssatz abgelegt. Die Größe des Operanden-Stacks hängt von den Operationen ab, die in der Methode durchgeführt

Operanden-Stack

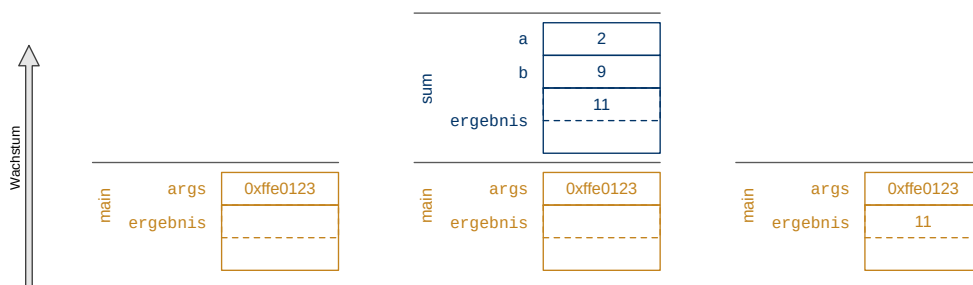
werden und wird vom Compiler berechnet. Entsprechend viel Platz wird im Aktivierungssatz für den Operanden-Stack reserviert. Zusätzlich berechnet der Compiler den benötigten Platz für die lokalen Variablen und Parameter und reserviert diesen ebenfalls für den Aktivierungssatz. (Vgl. Java-VM Spezifikation, Kapitel 2.6)

Die Rückgabe eines Wertes von einer Methode an den Aufrufer erfolgt indem der Rückgabewert auf den Operanden-Stack der aufrufenden Methode geschrieben wird. Hierzu dienen spezielle Byte-Code-Instruktionen (z. B. `areturn`). (Vgl. Java-VM Spezifikation, Kapitel 6.5)

14.6 Stack [219]

```
static long sum(int a, int b) {
    long ergebnis = a + b;
    return ergebnis;
}

public static void main(String[] args) {
    long ergebnis = sum(2, 9);
    System.out.println(ergebnis); // -> 11
}
```



Beispiel Stack-Wachstum

Vor dem Aufruf der `sum`-Methode liegt nur die `main`-Methode auf dem Stack (linkes Bild). Danach wird die `sum`-Methode aufgerufen und ein entsprechender Aktivierungssatz wird auf dem Stack erzeugt (mittleres Bild). Sobald die Methode zurückkehrt, wird das Ergebnis an die `main`-Methode zurückgegeben und der Aktivierungssatz der `sum`-Methode wird gelöscht.

Um die Darstellung in der Abbildung nicht zu kompliziert zu gestalten, wurde der Operanden-Stack nicht dargestellt.

14.7 Stack und Heap [220]

Auf dem Stack können in Java nur abgelegt werden

- Variablen primitiver Datentypen
- Referenzvariablen

Alle anderen Datentypen liegen auf dem Heap

- Strings
- Arrays
- Objekte

Objekt

- liegt immer auf dem Heap
- trägt Daten (primitive Datentypen oder Referenzen)

Referenz (reference) (Referenzvariable)

Referenz

- ist *kein* Objekt
- zeigt auf ein Objekt
- kann auf dem Heap liegen (Variable in einem Objekt)
- kann auf dem Stack liegen (lokale Variable)
- kann zu verschiedenen Zeiten auf verschiedene Objekte zeigen

14.8 Garbage Collection [222]

- Stack sorgt für Abräumen der lokalen Variablen
- Heap wird vom **Garbage Collector** aufgeräumt
 - ▶ durchläuft den Heap und sucht Objekte, die nicht mehr referenziert werden
 - ▶ entfernt alle unbenutzten Objekte

Garbage Collector

Index

Aktivierungssatz, 85
Aktualparameter, 75
ALGOL 60, 5
Assemblersprachen, 3
Assemblierer, 3
Assoziativität, 26
Attribute, 40
Attributen, 39
Ausdruck, 24
Ausdrücke, 15
Ausführen, 9
Ausgabeeanweisung, 11

BASIC, 6
Bedingungsoperator, 61
Binäre Operatoren, 24
boolean, 27
break;, 69

Camel-Notation, 18
Cast-Operator, 38
COBOL, 5
Compilation, 10
Compiler, 3, 9
Compilieren, 9
continue;, 69

Datentyp, 19, 20
Datentypen, 15
Defintion, 20
Deklaration, 20
Deklarative Programmierung, 1
Dekrement-Operator, 33
do-while-Schleife, 65

Editieren, 9
Eingabeeanweisung, 12
Elementare Datentypen, 27
Elemente, 46
escaped, 35, 51
Explitize Konvertierung, 38

for-Schleife, 66
foreach-Schleife, 68
Formalparameter, 75
FORTRAN, 4
Fragezeichen-Operator, 61
Funktion, 71

Ganzzahlen, 30
Garbage Collector, 87
Globale Variablen, 80

Heap, 84
Hochsprache, 3
höhere Programmiersprache, 3
höherer Datentyp, 46

if-Anweisung, 58
if-else-Anweisung, 58
if-else-if-Anweisung, 59
Imperative Programmierung, 1
Implizite Konvertierung, 38
Importanweisung, 12
Infinity, 37
Initialisierung, 15, 22
Inkrement-Operator, 33
Interpreter, 3

Java-Bytecode, 10

JavaDoc, 76
JDK, 13

Kellerspeicher, 84
Klasse, 39, 42
Konstante, 55
Kurzschlussoperatoren, 28

Laufvariable, 69
LIFO-Speicher, 84
links-assoziativ, 26
LISP, 4
Listen, 44
Literal, 25
Logische Operatoren, 28
Logische Vergleichsoperatoren, 29
lokalen Variablen, 78

Main-Klasse, 11
Main-Methode, 11
Maschinensprache, 2
Methode, 71
Methodendeklaration, 71
Mnemonic, 3

Name, 20
Namenskonvention, 18
NaN, 37

Objekt, 42
Operand, 24
Operanden-Stack, 85
Operator, 24
Ordinalzahlen, 30

Parameter, 71
Postfix-Operator, 33
primitive Datentypen, 27
primitiven Datentypen, 41
Programmierparadigma, 1
Programmiersprache, 2
Prozedur, 71
Präfix-Operator, 33
Punkt-Operator, 43, 52

Quelltext, 9

Rangfolge, 26

rechts assoziativ, 26
Referenz, 87
Referenzdatentypen, 41
Referenzvariable, 41
Referenzübergabe, 75
Rekursion, 82
Return-Anweisung, 73

Seiteneffekt, 81
Sichtbarkeit, 79
Signatur, 72
Stack, 84
Standardwerte, 48
Stapelspeicher, 84
Struktur, 39
switch-Anweisung, 60

Ternäre Operatoren, 24
ternärer Operator, 61

undefiniert, 22
uninitialisierte Variable, 22
Unäre Operatoren, 24

Varag-Methoden, 77
Variable, 17
Variablen, 17
Variablendefinition, 15
Variablendeklaration, 15
Verbund, 39
Verkettung, 52
Virtuellen Maschine, 10

Wert, 16
Wertübergabe, 75
while-Schleife, 64

Zeichenketten, 50
Zuweisung, 21
Zuweisungsoperator, 21

Überladene Methode, 77